

UNIVERSITÀ DEGLI STUDI DI MILANO
Facoltà di Scienze e Tecnologie
Corso di Laurea in Sicurezza dei Sistemi e delle Reti Informatiche

ANALISI PROBABILISTICA DEL
BLACKJACK: CONTEGGIO DELLE
CARTE E TECNICHE DI
RILEVAMENTO PER LA SICUREZZA
DEI SISTEMI LUDICI

Relatore: Prof. Davide Rocchesso

Correlatore:

Tesi di:
Samuele Bertoglio
Matricola: 31410A

Anno Accademico 2025-2026

dedicato a ...

Prefazione

hkjafgyruet.

Organizzazione della tesi

La tesi è organizzata come segue:

- nel Capitolo 1 si introduce il contesto e gli obiettivi del lavoro;
- nel Capitolo 2 si presentano i fondamenti matematici (probabilità condizionata, distribuzione ipergeometrica, processi di Markov, Teorema del Limite Centrale, metodi Monte Carlo) e i concetti di sicurezza nei sistemi ludici che verranno approfonditi nei capitoli successivi;
- nel Capitolo 3 il blackjack viene formalizzato come Processo Decisionale di Markov e validato tramite un simulatore Monte Carlo sviluppato per questa tesi;
- nel Capitolo 4 si analizza il sistema di conteggio Hi-Lo, la sua implementazione software e la validazione empirica del vantaggio statistico che offre al giocatore;
- nel Capitolo 5 si sviluppa un sistema di rilevamento automatico dei contatori basato sull'analisi comportamentale (**BehaviouralProfiler**) e si affrontano le minacce correlate di bot, collusion e multi-accounting;
- nel Capitolo 6 si analizzano le contromisure operative lato gioco e la sicurezza dell'infrastruttura informatica sottostante;
- nel Capitolo 7 si discutono gli aspetti etici, normativi e di privacy legati al rilevamento comportamentale, in particolare il GDPR;
- nell'Appendice A si approfondisce il tema della biometria comportamentale, di cui il **BehaviouralProfiler** costituisce un caso applicativo semplificato.

Capitolo 1

Introduzione

Capitolo 2

Fondamenta Matematiche e di Sicurezza

2.1 Fondamenti Matematici

2.1.1 Probabilità Condizionata e Teorema di Bayes

La probabilità condizionata esprime quanto sia probabile che un evento A si verifichi, sapendo che un altro evento B si è già verificato. Formalmente, dato $P(B) > 0$, si definisce:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

La quantità $P(A|B)$ viene detta probabilità a posteriori, poiché incorpora l'informazione fornita dall'evento B , mentre $P(A)$ è la probabilità a priori, calcolata in assenza di tale informazione. Il concetto chiave è che la conoscenza di un evento modifica la stima probabilistica degli eventi correlati.

Il Teorema di Bayes estende questo principio, consentendo di invertire la direzione della condizionalità: nota la probabilità $P(B|A)$, è possibile ricavare $P(A|B)$. La formula risultante è:

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

Il teorema non introduce nuovi assiomi, ma discende direttamente dalla definizione di probabilità condizionata e dall'additività della probabilità su eventi incompatibili.

Nel contesto del blackjack, questo principio trova applicazione diretta. Ogni carta estratta dal mazzo costituisce un nuovo evento che aggiorna la distribuzione delle carte rimanenti. Se nelle prime mani sono uscite molte carte basse, la probabilità che le successive siano alte, e quindi favorevoli al giocatore, aumenta in modo misurabile.

Il giocatore che conta le carte non fa altro che applicare implicitamente questo aggiornamento bayesiano mano per mano, stimando dinamicamente la composizione residua del mazzo sulla base delle informazioni progressivamente acquisite. Questo meccanismo verrà formalizzato nel Capitolo 3 attraverso il modello ‘Processo Decisionale di Markov’, e quantificato nel Capitolo 4 tramite il sistema ‘Hi-Lo’.

2.1.2 Distribuzione Ipergeometrica

La distribuzione ipergeometrica è una distribuzione di probabilità discreta che descrive l'estrazione senza reinserimento da una popolazione finita. A differenza della distribuzione binomiale, in cui ogni estrazione è indipendente dalla precedente, qui ogni carta estratta modifica la composizione residua della popolazione, introducendo dipendenza tra gli eventi successivi.

Formalmente, dato un insieme di N elementi di cui K appartengono a una categoria di interesse, la probabilità di ottenere esattamente k elementi di quella categoria estraendo un campione di dimensione n è:

$$P(X = k) = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}$$

Il valore atteso è $E[X] = n \cdot K/N$, che coincide con quello della distribuzione binomiale, ma la varianza risulta ridotta di un fattore correttivo dovuto all'assenza di reinserimento.

Nel blackjack, il mazzo si comporta esattamente come un'urna finita estratta senza reinserimento. Se un sabot da 6 mazzi contiene inizialmente 24 assi su 312 carte totali, la probabilità di ricevere un asso nella mano successiva non è costante ma dipende da quante carte, e di quale tipo, sono già state distribuite. Questa dipendenza statistica è il fondamento matematico del conteggio delle carte, e giustifica perché l'approssimazione con mazzo infinito (distribuzione binomiale) sia valida solo per basse penetrazioni del mazzo.

2.1.3 Processi di Markov(MDP)

Un Processo Decisionale di Markov (MDP) è un modello matematico formale per rappresentare problemi decisionali in ambienti stocastici, ovvero in cui l'esito di un'azione è parzialmente determinato dal caso. Un MDP è descritto dalla quadrupla (S, A, P, R) , dove S è lo spazio degli stati, A l'insieme delle azioni disponibili, $P(s' | s, a)$ la funzione di transizione che descrive la probabilità di passare dallo stato s allo stato s' eseguendo l'azione a , e $R(s, a)$ la funzione di ricompensa associata.

La proprietà fondamentale che caratterizza un processo markoviano è che lo stato futuro dipende unicamente dallo stato corrente e dall'azione intrapresa, e non dalla sequenza di stati precedenti. Questa proprietà, nota come *proprietà di Markov*, semplifica notevolmente l'analisi formale del sistema.

L'obiettivo di un MDP è identificare una politica ottima π^* , ovvero una funzione che associa a ogni stato l'azione che massimizza il valore atteso cumulativo delle ricompense future:

$$\pi^*(s) = \arg \max_{a \in A} \mathbb{E}[R \mid s, a]$$

Il blackjack si presta naturalmente a questa formalizzazione. Lo stato del sistema può essere definito minimalmente come la coppia (h_p, c_b) , il totale della mano del giocatore e la carta scoperta del banco, oppure esteso includendo la composizione residua del mazzo D . In quest'ultimo caso il sistema diventa formalmente markoviano, ma lo spazio degli stati cresce in modo significativo. La Strategia di Base del blackjack è precisamente la politica ottima derivata sotto l'assunzione di mazzo con composizione neutra.

2.1.4 Teorema del Limite Centrale

Il Teorema del Limite Centrale (TLC) afferma che la distribuzione della media di un gran numero di variabili casuali indipendenti e identicamente distribuite tende a una distribuzione normale, indipendentemente dalla distribuzione di partenza. Formalmente, date n variabili $X_1, X_2, \dots, X_n$ con media μ e varianza σ^2 finite, la media campionaria standardizzata converge in distribuzione alla normale standard:

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1)$$

Nel caso delle simulazioni sviluppate in questa tesi, ogni mano può essere trattata come una variabile aleatoria X_i indipendente dalle precedenti e con la stessa distribuzione: a mazzo rimescolato, le probabilità si reinizializzano e l'esito di una mano non condiziona quello delle successive. È proprio questa struttura i.i.d. che rende applicabile il TLC e garantisce la convergenza della media campionaria al valore atteso teorico.

Nel contesto del blackjack e delle simulazioni sviluppate in questa tesi, il TLC non è solo un concetto teorico, ma lo strumento che permette di quantificare l'affidabilità dei risultati. Poiché l'esito di una singola mano è una variabile discreta con alta varianza, è solo attraverso il TLC che possiamo garantire che la media di milioni di mani simulate converga verso il **valore atteso** (*Expected Value*) teorico del gioco.

Questo teorema giustifica l'utilizzo dell'errore standard ($SE = \sigma/\sqrt{n}$) come misura della precisione: ci permette di affermare che, all'aumentare delle prove, la stima

dell'house edge diventa estremamente precisa, seguendo una curva gaussiana centrata sul valore reale. Senza questo presupposto, non sarebbe possibile distinguere un reale vantaggio statistico dal rumore di una semplice fluttuazione fortunata del campione.

2.1.5 Metodi Monte Carlo

Il metodo Monte Carlo è un'ampia classe di metodi computazionali basati sul campionamento casuale per ottenere risultati numerici. L'idea fondamentale è semplice: anziché calcolare analiticamente una quantità complessa, si simulano un gran numero di esperimenti casuali e si stima il valore di interesse come media dei risultati ottenuti.

Le origini del metodo risalgono alla metà degli anni Quaranta, nell'ambito del Progetto Manhattan. I principali formalizzatori furono Enrico Fermi, John von Neumann e Stanisław Ulam. Il nome *Monte Carlo* fu coniato da Nicholas Metropolis in riferimento al celebre casinò.

Il principio matematico sottostante è la Legge dei Grandi Numeri: data una variabile casuale X con valore atteso μ , la media campionaria di n realizzazioni indipendenti converge al valore teorico:

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{P} \mu \quad \text{per } n \rightarrow \infty$$

L'errore della stima Monte Carlo diminuisce come $\sigma/\sqrt{n}$: per dimezzare l'errore occorre quadruplicare il numero di simulazioni. Questo tasso di convergenza è indipendente dalla complessità del problema, il che rende il metodo particolarmente adatto a sistemi con molte variabili o senza soluzione analitica chiusa.

Nel contesto di questa tesi il metodo Monte Carlo costituisce il principale strumento di validazione empirica. Le simulazioni eseguite su 5 milioni di mani per ciascuna strategia, sfruttano direttamente questo principio: ogni mano rappresenta un esperimento casuale indipendente, e la media cumulativa dei payoff converge all'house edge teorico con un errore standard stimabile e controllato.

Il simulatore implementa il metodo Monte Carlo iterando un numero arbitrario di mani, gestendo il reshuffle automatico a penetrazione fissa e raccogliendo le statistiche per ogni risultato:

```
for hand_num in range(num_hands):
    # reshuffle a 85% di penetrazione
    if game.deck.cards_remaining() < 0.15 * (52 * self.
num_decks):
        game.deck = Deck(self.num_decks, rng=self.rng)
        if hasattr(self.strategy, 'reset_count'):
            self.strategy.reset_count()
```

```

base_bet = self.strategy.get_bet_size() \
    if hasattr(self.strategy, 'get_bet_size') else 1

game.deal_initial_cards(self.strategy)
game.player_turn(self.strategy)
game.dealer_turn(self.strategy)
results = game.determine_outcome()

for idx, result in enumerate(results):
    if result == 'win':
        payoff = 1.5 * bet if hand.is_blackjack() else bet
    elif result == 'loss':
        payoff = -bet
    else:
        payoff = 0
    total_payoff += payoff

```

La struttura è parametrica rispetto alla strategia e al generatore di numeri casuali: la stessa classe `MonteCarloSimulator` viene utilizzata per la `AdvancedStrategy`, la `CardCountingStrategy` e le varianti di camouflage analizzate nei capitoli successivi.

2.2 Sicurezza nei Sistemi Ludici

I principi che seguono si applicano ai sistemi ludici in generale; la loro declinazione specifica nel contesto online e digitale viene approfondita nei Capitoli 5 e 6, dove il modello teorico viene calato nell'infrastruttura informatica concreta.

2.2.1 Minacce Tipiche: bot, multi-accounting, collusion

I sistemi ludici sono esposti a una varietà di minacce che minano l'integrità del gioco, la sicurezza degli utenti e l'equilibrio economico delle piattaforme. Le principali si articolano in tre categorie.

I *bot* sono software automatizzati che simulano il comportamento di un giocatore umano con l'obiettivo di ottenere vantaggi illeciti in modo sistematico. In giochi come il blackjack, un bot può applicare la strategia ottimale in modo infallibile e senza errori, oppure eseguire il conteggio delle carte con precisione assoluta, comportamenti impossibili per un giocatore umano nel lungo periodo. I bot sono rilevabili attraverso anomalie nei tempi di risposta, assenza di varianza decisionale e pattern di puntata meccanici, elementi rilevabili statisticamente e modellati nei moduli `botDetection.py`, `multiAccountingDetection.py` e `collusionDetection.py`, descritti nel Capitolo 5.

Il *multi-accounting* consiste nella registrazione di più account da parte dello stesso utente, spesso per aggirare un ban già ricevuto, accumulare bonus riservati ai nuovi iscritti, o occupare più posizioni ai tavoli contemporaneamente. È una delle minacce più difficili da contrastare perché non richiede competenze tecniche avanzate.

La *collusion* (collusione) è una forma di frode cooperativa in cui più giocatori si coordinano per manipolare l'esito di una partita a vantaggio di uno di loro. In questa tecnica, nota anche come *chip dumping*, un giocatore si unisce a una partita da più account, aumentando le probabilità di vincere poiché gli altri account perdono deliberatamente. Nei giochi multiplayer questa pratica altera direttamente la fairness del sistema.

Queste minacce condividono una caratteristica comune: si manifestano come comportamento anomalo rilevabile statisticamente, il che le rende candidate naturali per sistemi di detection automatica come quelli descritti nel Capitolo 5.

2.2.2 RNG: PRNG vs CSPRNG

Un generatore di numeri casuali (RNG, *Random Number Generator*) è un componente software che produce sequenze di valori non predicibili, utilizzato nei sistemi ludici per determinare l'ordine di distribuzione delle carte o qualsiasi altro evento aleatorio. La qualità del generatore determina direttamente l'equità del gioco.

I PRNG (*Pseudo-Random Number Generator*) sono algoritmi deterministici che producono sequenze numeriche apparentemente casuali, ma completamente determinate da un valore iniziale detto seed. L'algoritmo Mersenne Twister, utilizzato nel modulo `random` di Python, è il PRNG più diffuso in ambito scientifico: ha un periodo di $2^{19937} - 1$ ed è statisticamente eccellente per simulazioni. Tuttavia, dato il seed, l'intera sequenza futura è predicibile.

I CSPRNG (*Cryptographically Secure Pseudo-Random Number Generator*) sono progettati specificamente per resistere a questo tipo di attacco. A differenza dei PRNG, anche conoscendo parte della sequenza prodotta, è computazionalmente impossibile prevedere i valori futuri o ricostruire lo stato interno del generatore. Le implicazioni di sicurezza di questa distinzione, incluse le vulnerabilità legate alla prevedibilità del seed, vengono approfondite nel Capitolo 6.

Nei sistemi ludici il CSPRNG è necessario ogni volta che l'imprevedibilità ha un valore economico diretto. Il PRNG rimane appropriato per simulazioni offline dove l'obiettivo è la validità statistica e non la resistenza ad attacchi. Questa distinzione è implementata concretamente nel simulatore sviluppato per questa tesi, come descritto nella sezione 3.4.

2.2.3 Privacy e Tracking

I sistemi ludici raccolgono spesso quantità significative di dati personali: dati anagrafici, storico delle sessioni, cronologia delle puntate e, nei sistemi più avanzati, segnali biometrici comportamentali. Questa raccolta è necessaria per funzionalità legittime come la verifica dell'identità e la prevenzione delle frodi, ma genera al contempo rischi rilevanti per la privacy degli utenti.

Il quadro normativo di riferimento per i sistemi operanti in ambito europeo è il Regolamento Generale sulla Protezione dei Dati (GDPR, Reg. UE 2016/679) [13]. Il GDPR impone in particolare il principio di *data minimization*, raccogliere solo i dati strettamente necessari, e richiede una valutazione preventiva dell'impatto quando i trattamenti comportano rischi elevati per i diritti degli utenti.

Un aspetto particolarmente delicato riguarda i sistemi di rilevamento automatico dei comportamenti anomali, come quelli descritti nel Capitolo 5. Questi sistemi raccolgono e analizzano continuamente il comportamento degli utenti durante la sessione, costruendo profili dettagliati. Benché la finalità sia legittima, il confine con la profilazione commerciale è sottile e richiede un'attenzione normativa specifica.

2.2.4 Standard e Normative

Il settore del gioco d'azzardo è soggetto a un doppio livello di regolamentazione: quello sovranazionale e quello nazionale.

A livello europeo il riferimento principale è il GDPR per la protezione dei dati, affiancato dalla Direttiva sui Servizi (2006/123/CE) e dalle raccomandazioni della Commissione Europea sul gioco responsabile. L'assenza di una direttiva europea armonizzata specifica per il gioco d'azzardo online lascia agli Stati membri ampia autonomia regolamentare, producendo un panorama normativo frammentato.

In Italia, l'autorità di settore è l'ADM (Agenzia delle Dogane e dei Monopoli), che rilascia le licenze agli operatori autorizzati e supervisiona il rispetto delle normative tecniche e di sicurezza. Il GDPR ha imposto l'obbligo di notificare eventuali violazioni dei dati entro 72 ore, aumentando la pressione sugli operatori con licenza ADM per mantenere una vigilanza costante sulle proprie infrastrutture.

Sul piano tecnico, gli standard di riferimento per la certificazione degli RNG e dei sistemi di gioco sono quelli emanati da enti come eCOGRA e BMM Testlabs, che verificano la correttezza statistica degli algoritmi e l'equità dei sistemi prima della messa in produzione.

Capitolo 3

Il Blackjack come Modello di Sistema

3.1 Formalizzazione del Gioco

Il blackjack è una sfida individuale tra giocatore e banco. L'obiettivo è battere il punteggio del banco senza mai superare il valore di 21, fase chiamata "sballare". Si utilizzano uno o più mazzi di carte francesi da 52 carte ciascuno; i semi (cuori, quadri, fiori, picche) sono irrilevanti, e conta solo il valore numerico o convenzionale di ogni carta. Il giocatore vince realizzando un punteggio superiore a quello del banco, mentre subisce una sconfitta se ottiene un punteggio inferiore o eccede il limite di 21.

Il valore numerico di ciascuna carta è definito in modo convenzionale: le carte numeriche dalla 2 alla 9 valgono il loro valore nominale, le figure (J, Q, K) e il 10 valgono ciascuna 10, mentre l'Asso (A) può valere 1 oppure 11 a seconda del contesto. Una mano contenente un Asso conteggiato come 11 senza causare *bust* viene definita *soft hand* (es. A-6 = soft 17), mentre una mano priva di Asso, o in cui l'Asso deve essere contato come 1, è definita *hard hand* (es. 10-7 = hard 17). Il blackjack naturale si verifica quando le prime due carte distribuite totalizzano 21 (Asso + carta di valore 10): questa combinazione costituisce la mano migliore possibile ed è ricompensata con un pagamento maggiorato nel rapporto 3:2.

A ogni turno il giocatore può scegliere tra quattro azioni. Con **Hit** riceve un'ulteriore carta dal mazzo, azione ripetibile fino a quando non decide di fermarsi o supera 21. Con **Stand** termina il proprio turno mantenendo il totale corrente. Con **Double Down** raddoppia la puntata iniziale ricevendo esattamente una carta aggiuntiva, dopodiché il turno termina automaticamente. Infine, con **Split**, disponibile solo se le prime due carte hanno lo stesso valore, le divide in due mani separate, ciascuna con una puntata indipendente, da giocare separatamente.

Il banco opera secondo regole deterministiche e non esercita alcuna autonomia decisionale. Nella configurazione standard deve chiedere carta (*hit*) se il proprio totale è inferiore a 17, e fermarsi (*stand*) se è pari o superiore a 17. Esiste una variante in cui il banco deve continuare a pescare anche con soft 17 (Asso contato come 11 più 6), regola che modifica il vantaggio matematico complessivo del gioco. Il comportamento deterministico del banco elimina ogni componente strategica dal suo lato, rendendo il modello più semplice da analizzare rispetto ad altri giochi competitivi.

Una mano di blackjack si sviluppa in cinque fasi sequenziali. Nella **distribuzione iniziale** il banco distribuisce due carte a ciascun giocatore e due a se stesso, lasciando una carta scoperta (*upcard*) e una coperta (*hole card*). Segue la **verifica del blackjack naturale**: se il giocatore o il banco raggiungono 21 con le prime due carte, la mano termina immediatamente. Nel **turno del giocatore** questi sceglie le proprie azioni in sequenza fino a fermarsi o il totale supera il 21. Nel **turno del banco** la carta coperta viene rivelata e le regole deterministiche vengono applicate. Infine, nella fase di **risoluzione**, si confrontano i totali e si determina l'esito: vittoria, sconfitta o pareggio.

Il sistema di ricompense presenta un'asimmetria fondamentale. Il blackjack naturale del giocatore è ricompensato con $+1.5 \times$ la puntata (rapporto 3:2), la vittoria standard con $+1 \times$ la puntata (1:1), il pareggio (*push*) con 0, e la sconfitta o il bust con $-1 \times$ la puntata. L'asimmetria risiede nel fatto che se il giocatore va in *bust* perde immediatamente, indipendentemente dal successivo risultato del banco, mentre il blackjack naturale è ricompensato in modo maggiorato rispetto alla semplice vittoria.

Esito	Payoff
Blackjack naturale del giocatore	$+1.5 \times$ puntata (3:2)
Vittoria standard	$+1 \times$ puntata (1:1)
Pareggio (<i>push</i>)	0
Sconfitta	$-1 \times$ puntata
Bust del giocatore	$-1 \times$ puntata (perdita immediata)
Blackjack dopo split	$+1 \times$ puntata (1:1, non naturale)

3.2 Modellazione come Processo Decisionale di Markov (MDP)

Di seguito si fornisce la declinazione al blackjack della quadrupla (S, A, P, R) introdotta in Sezione 2.1.3. Uno stato del sistema può essere definito in modo minimale o esteso, a seconda del livello di dettaglio richiesto. Nella forma minimale lo stato è la coppia $s = (h_p, c_b)$, dove h_p è il totale della mano del giocatore e c_b è la carta scoperta del banco: questo modello assume indipendenza dalla composizione del mazzo, approssimando il sistema come estratto da un mazzo “infinito” con probabilità costanti. Nella forma

estesa lo stato diventa $s = (h_p, c_b, D)$, dove D rappresenta la composizione residua del mazzo, ovvero il numero di carte rimanenti per ciascun valore. Includere D rende il sistema formalmente markoviano, poiché la probabilità futura dipende solo dallo stato corrente, ma aumenta drasticamente la dimensionalità dello spazio degli stati.

L'insieme delle azioni disponibili $A = \{\text{Hit}, \text{Stand}, \text{Double}, \text{Split}\}$ dipende dallo stato corrente: *Split* è possibile solo se le prime due carte hanno lo stesso valore, mentre *Double* è spesso limitato alle prime due carte. Il banco non dispone di un insieme di azioni: la sua politica è deterministica e fissata dalle regole.

La funzione $P(s' | s, a)$ descrive come lo stato evolve in base all'azione scelta. Se lo stato corrente è $s = (16, 7)$, il giocatore ha 16 e il banco mostra 7, e l'azione è *Hit*, allora $P(s' | s, \text{Hit})$ corrisponde alla distribuzione delle carte rimanenti nel mazzo: se la carta estratta è un 5, il nuovo stato sarà $s' = (21, 7)$; se è un 7, il giocatore va in *bust* e la mano termina.

La politica ottima π^* , definita in Sezione 2.1.3, individua per ogni stato l'azione che massimizza il valore atteso cumulativo delle ricompense future. La cosiddetta Strategia di Base del blackjack è la politica ottima derivata assumendo un mazzo con composizione neutra. In letteratura è ampiamente dimostrato che la sua applicazione perfetta in un tavolo con regole standard (6 mazzi, banco si ferma su *soft 17*) riduce il vantaggio del banco a un valore compreso tra lo 0.4% e lo 0.5%, come illustrato nella Figura 1. Questo valore, noto come **house edge teorico**, rappresenta il limite asintotico a cui le simulazioni dovranno tendere per confermare la validità del modello software sviluppato (si veda la Sezione 3.2.1 per la derivazione formale).

Mano del giocatore		Carta scoperta del banco									
		2	3	4	5	6	7	8	9	10	Asso
Le carte del giocatore	8	C	C	C	C	C	C	C	C	C	C
	9	C	R	R	R	R	C	C	C	C	C
	10	R	R	R	R	R	R	R	R	C	C
	11	R	R	R	R	R	R	R	R	R	C
	12	C	C	C	S	S	S	C	C	C	C
	13	S	S	S	S	S	S	C	C	C	C
	14	S	S	S	S	S	S	C	C	C	C
	15	S	S	S	S	S	S	C	C	C	C
	16	S	S	S	S	S	S	C	C	C	C
	17	S	S	S	S	S	S	S	S	S	S
	A-2	C	C	C	C	R	R	C	C	C	C
	A-3	C	C	C	C	R	R	C	C	C	C
	A-4	C	C	C	R	R	R	C	C	C	C
	A-5	C	C	C	R	R	R	C	C	C	C
	A-6	C	R	R	R	R	R	C	C	C	C
	A-7	S	R	R	R	R	R	S	S	S	S
	A-8	S	S	S	S	S	S	S	S	S	S
A-9	S	S	S	S	S	S	S	S	S	S	
A-A	D	D	D	D	D	D	D	D	D	D	
10-10	S	S	S	S	S	S	S	S	S	S	
9-9	D	D	D	D	D	S	D	D	S	S	
8-8	D	D	D	D	D	D	D	D	D	D	
7-7	D	D	D	D	D	D	C	C	C	C	
6-6	D	D	D	D	D	C	C	C	C	C	
5-5	R	R	R	R	R	R	R	R	C	C	
4-4	C	C	C	C	D	D	C	C	C	C	
3-3	D	D	D	D	D	D	C	C	C	C	
2-2	D	D	D	D	D	D	C	C	C	C	
Mano del giocatore		2	3	4	5	6	7	8	9	10	Asso
Carta scoperta del banco											

Figura 1: Tabella della Strategia di Base del blackjack per configurazione a 6 mazzi.

Nella tabella, ogni cella indica l'azione ottimale in funzione della mano del giocatore (righe) e della carta scoperta del banco (colonne). Le abbreviazioni utilizzate sono: **C** = Hit (chiedere carta), **S** = Stand (stare), **R** = Double Down (raddoppiare), **D** = Split (dividere). Nelle righe relative alle coppie, l'azione **D** indica lo split della coppia, mentre nelle righe delle mani soft indica il double down.

3.2.1 House Edge Teorico della Strategia di Base

Il valore teorico dell'house edge non è arbitrario: fu calcolato per la prima volta da Baldwin et al. nel 1956 [1], che dimostrarono per via analitica l'esistenza di una strategia ottimale per il blackjack e ne quantificarono il costo residuo per il giocatore.

Applicando la politica ottima π^* derivata nella sezione precedente, il valore atteso per mano è definito come:

$$EV = \sum_{s \in S} P(s) \cdot R(\pi^*(s))$$

dove $P(s)$ è la probabilità stazionaria di trovarsi nello stato s e $R(\pi^*(s))$ è la ricompensa attesa applicando l'azione ottimale. La valutazione esatta di questa somma richiede l'enumerazione completa dello spazio degli stati sotto l'assunzione di composizione neutra del mazzo.

Per la configurazione standard utilizzata in questa tesi, 6 mazzi, banco si ferma su qualsiasi 17 (*stand on all 17s*), double consentito sulle prime due carte, i calcoli analitici condotti da Griffin [5] e Wong [4] producono un house edge compreso tra **+0.45%** e **+0.52%** a favore del banco. Questo intervallo costituisce il benchmark teorico di riferimento: qualsiasi simulazione Monte Carlo sufficientemente lunga su questa configurazione deve convergere a un house edge in questo range per essere considerata valida. È precisamente a questo valore che si farà riferimento nella Sezione 3.5 quando i risultati simulati vengono definiti “coerenti con i valori teorici”: la coerenza è verificabile perché il benchmark è ora derivato e motivato formalmente, non semplicemente assunto come noto.

3.3 Proprietà Markoviane e memoria del sistema

Il blackjack presenta una caratteristica peculiare rispetto ad altri giochi aleatori: l'estrazione senza reinserimento. A differenza del lancio di dadi o della roulette, dove ogni evento è statisticamente indipendente, nel blackjack la composizione del mazzo cambia dinamicamente a ogni carta estratta, introducendo una dipendenza storica tra gli eventi. Se lo stato viene definito senza includere D , il sistema perde formalmente la proprietà di Markov, poiché la probabilità di transizione dipende dalla storia delle carte estratte; includendo invece D , il sistema diventa formalmente markoviano, ma lo spazio degli stati cresce in modo significativo, rendendo l'analisi analitica impraticabile.

Questa caratteristica rende il blackjack un modello didatticamente rilevante per studiare la probabilità condizionata, l'aggiornamento dinamico delle credenze in ottica bayesiana, e il compromesso tra complessità del modello e accuratezza della rappresentazione.

3.4 Aspetti informatici: simulazioni e RNG

3.4.1 Architettura del simulatore

Il sistema di simulazione è stato implementato in Python 3.11 con un approccio orientato agli oggetti, organizzato in moduli distinti secondo una gerarchia a tre livelli: entità di gioco, logica di simulazione e analisi/sicurezza.

Livello Base: Entità di Gioco

Al livello base, i moduli `card.py`, `deck.py` e `hand.py` modellano le entità elementari del gioco, come dettagliato nella Tabella 1.

Modulo	Funzione e Dettagli Implementativi
<code>card.py</code>	Contiene la classe <code>Card</code> , che rappresenta una singola carta tramite gli attributi <code>rank</code> e <code>suit</code> , ed espone il metodo <code>get_value()</code> per restituire il valore numerico secondo le regole del blackjack.
<code>deck.py</code>	Definisce la classe <code>Deck</code> per la gestione del mazzo (o della scarpa). Accetta <code>num_decks</code> come parametro, mantiene l'attributo <code>cards_remaining</code> per tracciare le carte residue e offre i metodi <code>shuffle()</code> e <code>draw()</code> .
<code>hand.py</code>	Implementa la classe <code>Hand</code> per rappresentare una mano di gioco. Calcola il punteggio totale risolvendo automaticamente l'ambiguità del valore dell'Asso ed espone i metodi <code>is_bust()</code> e <code>is_blackjack()</code> per la gestione degli stati terminali.

Tabella 1: Moduli del livello base: entità di gioco.

Livello Intermedio: Logica di Simulazione

Al livello intermedio, il nucleo operativo coordina lo svolgimento delle partite e l'applicazione delle strategie, come dettagliato nella Tabella 2.

Modulo	Funzione e Dettagli Implementativi
<code>game.py</code>	La classe Game coordina la logica della partita, gestendo la distribuzione iniziale, il turno del giocatore (inclusi split e double), il turno del banco e la determinazione dell'esito finale.
<code>rng.py</code>	Incapsula la generazione di numeri casuali in due implementazioni distinte: rng_PRNG , basata sull'algoritmo Mersenne Twister per le simulazioni offline, e rng_CSPRNG , basata sul modulo secrets per i contesti in cui l'imprevedibilità ha un valore economico diretto. La distinzione teorica tra i due generatori è discussa nel Capitolo 2, mentre le implicazioni di sicurezza sono analizzate nel Capitolo 6.
<code>strategy.py</code> e <code>cardCountingStrategy.py</code>	Definiscono le strategie di gioco, entrambe esposte tramite il metodo standardizzato decide(game_state) . Il primo modulo implementa la classe AdvancedStrategy (strategia ottimale senza conteggio); il secondo implementa CardCountingStrategy , basata sul sistema Hi-Lo con calcolo del True Count, bet spread 1-2-4-12 e deviazioni strategiche (illustrate nel Capitolo 4).
<code>montecarlo.py</code>	Il motore di simulazione vero e proprio (MonteCarloSimulator) gestisce una penetrazione del mazzo impostata al 85% e traccia i vettori tc_history e bet_history per l'analisi comportamentale. Una scelta progettuale rilevante riguarda l'ordine delle operazioni: la puntata viene registrata in bet_history <i>prima</i> della distribuzione delle carte. Tale flusso rispecchia fedelmente il comportamento reale di un contatore, il quale decide l'entità della puntata sulla base del True Count accumulato nelle mani precedenti e non in funzione delle carte che sta per ricevere.

Tabella 2: Moduli del livello intermedio: logica di simulazione.

Livello Superiore: Analisi e Sicurezza

Al livello superiore, i moduli sono separati e organizzati per finalità specifiche di validazione metrologica, rilevamento delle minacce e testing architetturale, come dettagliato nella Tabella 3.

Modulo	Funzione e Dettagli Implementativi
<code>convergenza.py</code>	Verifica la convergenza dell'errore standard alla legge $1/\sqrt{n}$ tramite la generazione di plot log-log.
<code>run_simulations.py</code>	Esegue il confronto tra <code>AdvancedStrategy</code> e <code>CardCountingStrategy</code> su un campione di 5 milioni di mani per la validazione statistica rispetto alla letteratura.
<code>behaviouralProfiler.py</code>	Implementa la classe <code>BehaviouralProfiler</code> per il rilevamento di anomalie comportamentali e pattern di conteggio (Capitolo 5).
<code>TcamouflageTest.py</code>	Simula l'applicazione di strategie di evasione e mascheramento da parte del giocatore per aggirare i sistemi di controllo.
<code>clientServerSim.py</code>	Modella un'architettura client-server per valutare la resilienza del protocollo a replay attack e manipolazioni client-side.
<code>seedPredictionAttack.py</code>	Dimostra la vulnerabilità e il determinismo del PRNG standard, validando l'efficacia della mitigazione tramite CSPRNG (Capitolo 6).
<code>TEST_basico.py</code>	Raccoglie i test unitari e di integrazione end-to-end sulla <code>CardCountingStrategy</code> (validazione Hi-Lo, calcolo True Count, correttezza del bet sizing e deviazioni).
<code>botDetection.py</code>	Rileva software automatizzati tramite analisi di T_{avg} e T_{var} : tempi inferiori a 1 s con varianza sotto 0.05 segnalano comportamento meccanico non umano.
<code>multiAccountingDetection.py</code>	Confronta il vettore comportamentale $[T_{avg}, T_{var}, B_{avg}]$ tra sessioni di account diversi: distanza euclidea inferiore a 0.5 indica profili compatibili con lo stesso utente.
<code>collusionDetection.py</code>	Calcola la correlazione di Pearson tra le puntate di due giocatori allo stesso tavolo: correlazione negativa forte ($r < -0.7$) segnala chip dumping deliberato (Capitolo 5).

Tabella 3: Moduli di analisi e sicurezza del simulatore.

3.4.2 Random Number Generator (RNG)

La scelta del generatore di numeri casuali ha implicazioni dirette sulla qualità e sulla sicurezza delle simulazioni. Il PRNG standard adottato è il Mersenne Twister (`rng_PRNG`): deterministico e riproducibile tramite seed, è ideale per simulazioni offline dove l'obiettivo è la validità statistica. Il suo limite è la prevedibilità: dato il seed, l'intera sequenza futura è determinata. Per i contesti in cui l'imprevedibilità ha valore economico si utilizza `rng_CSPRNG`, basato su `secrets`, che attinge all'entropia del sistema operativo rendendo computazionalmente impossibile anticipare i valori futuri. Le proprietà crittografiche di questa distinzione e le relative vulnerabilità sono approfondite nel Capitolo 6.

3.4.3 Simulazioni Monte Carlo

Il metodo Monte Carlo si basa sulla ripetizione di esperimenti casuali per stimare quantità di interesse. In questa tesi ogni mano rappresenta un esperimento indipendente il cui esito economico X_i vale +1 in caso di vittoria, -1 in caso di sconfitta e 0 in caso di pareggio. La stima del valore atteso è:

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n X_i \quad SE = \frac{\sigma}{\sqrt{n}}$$

Le simulazioni sono state condotte sulla configurazione standard a 6 mazzi, con banco che si ferma su qualsiasi 17 (*stand on all 17s*), campione di $n = 5 \cdot 10^6$ mani per ciascuna strategia. La variazione sistematica del numero di mazzi $k \in \{1, 2, 4, 6, 8\}$ e della regola del soft 17 rappresenta una direzione naturale per approfondimenti futuri, ma non è stata inclusa nelle simulazioni principali al fine di mantenere i risultati confrontabili con un unico benchmark teorico di riferimento. I risultati mostrano la convergenza delle stime al crescere di n , come riportato nella tabella seguente.

n	Win Rate	House Edge	Std Error	Tempo (s)
1,000	0.4158	0.037332	0.034129	0.05
10,000	0.4275	0.013136	0.010933	0.13
100,000	0.4353	-0.001114	0.003446	1.71
500,000	0.4310	0.002359	0.001541	9.35
1,000,000	0.4331	0.000355	0.001088	14.02
5,000,000	0.4331	0.003597	0.000487	308.73

3.5 Risultati delle Simulazioni Monte Carlo

Dalle simulazioni eseguite emergono tre andamenti principali. L'house edge converge verso valori compresi tra 0.006 e 0.008 (circa 0.6–0.8%) per la **AdvancedStrategy** nella configurazione senza surrender, e risulta negativo per la **CardCountingStrategy** con spread 1-12, risultando coerente con i valori teorici derivati nella Sezione 3.2.1. L'errore standard decresce seguendo la legge $SE \propto 1/\sqrt{n}$, confermato da un fit log-log con coefficiente di determinazione $R^2 > 0.99$. La stabilità numerica è raggiunta per $n \geq 10^6$, con un errore relativo inferiore allo 0.1%.

Per la validazione della strategia avanzata è stato scelto un campione di $n = 10^6$ mani, ottenendo un $SE \approx 0.001$. Per la **CardCountingStrategy**, dove il vantaggio atteso può diventare positivo ma presenta una volatilità molto più elevata, il numero di simulazioni è stato incrementato a $5 \cdot 10^6$ mani, per garantire che il segnale del profitto fosse statisticamente distinguibile dal rumore della varianza.

3.5.1 Verifica Empirica della Legge $\sqrt{n}$

La figura 2 mostra la convergenza dell'errore standard al crescere del numero di simulazioni. I dati empirici seguono fedelmente il modello teorico $SE = k/\sqrt{n}$ con coefficiente di determinazione $R^2 > 0.99$, confermando la validità della legge dei grandi numeri. Si osserva inoltre che raddoppiare la precisione richiede quadruplicare il numero di simulazioni: per ridurre l'errore dall'1% allo 0.5% è necessario passare da 10,000 a 40,000 mani.

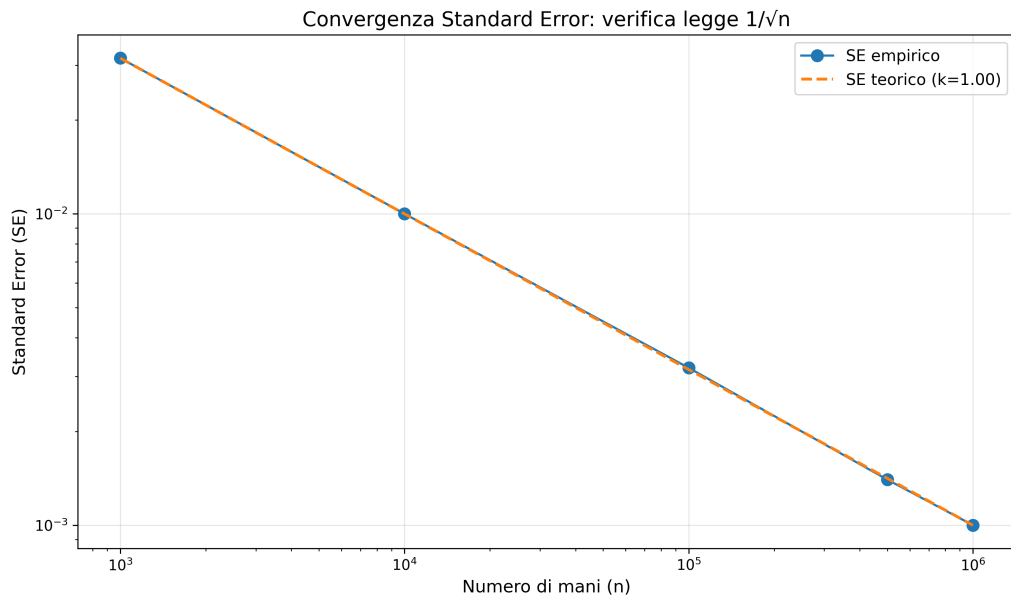


Figura 2: Convergenza dell'errore standard al crescere del numero di simulazioni. Il fit log-log conferma $R^2 > 0.99$.

3.5.2 Dipendenza Storica e Opportunità di Conteggio

A differenza della roulette o del lancio di dadi, dove ogni evento è statisticamente indipendente, nel blackjack l'estrazione senza reinserimento fa sì che la composizione residua del mazzo cambi dinamicamente, introducendo una *dipendenza storica* tra gli eventi. Dal punto di vista della fairness matematica questo non costituisce un problema: ogni carta ha probabilità $P(\text{carta}) = \text{carte rimanenti} / \text{totale carte rimanenti}$ e, per la Legge dei Grandi Numeri, i risultati convergono comunque verso il valore atteso teorico.

Tuttavia, questa caratteristica introduce un'opportunità strategica. Un giocatore che traccia sistematicamente le carte uscite può stimare dinamicamente la composizione residua del mazzo e adattare di conseguenza puntate e decisioni. Edward O. Thorp fu

il primo a formalizzare questo principio nel 1962, dimostrando in *Beat the Dealer* [2] che la variazione della distribuzione delle carte poteva modificare il vantaggio statistico tra giocatore e banco. La tecnica che ne deriva, il *conteggio delle carte*, sfrutta questa informazione per costruire una stima probabilistica in tempo reale: ogni punto di incremento nel *True Count* sposta il vantaggio verso il giocatore di circa lo 0.5%, consentendo nel lungo periodo di invertire un house edge che, con la sola strategia avanzata, si attesta attorno allo 0.4–0.5%.

Il Capitolo 4 analizza nel dettaglio il sistema Hi-Lo, quantificando questo vantaggio in funzione del *True Count* e della penetrazione del mazzo. I Capitoli 5 e 6 ne esplorano le implicazioni per la sicurezza dei sistemi ludici online, in particolare come rilevare i giocatori che applicano queste tecniche e quali contromisure adottare per preservare l'equilibrio economico del gioco.

Capitolo 4

Il Sistema Hi-Lo e il Conteggio delle Carte

4.1 Classificazione dei Sistemi di Conteggio

I sistemi di conteggio delle carte si classificano in base alla complessità dei valori assegnati alle carte. I sistemi Level 1 (± 1), tra cui Hi-Lo, KO e Red Seven, usano soli tre valori e minimizzano il rischio di errore umano. I sistemi Level 2 (± 2), come Hi-Opt II, Omega II e Zen, aumentano la precisione a scapito della semplicità. I sistemi Level 3 o con valori frazionari, come Wong Halves, rappresentano il limite teorico dell'efficacia ma sono impraticabili in condizioni reali. Il sistema Hi-Lo raggiunge il 97% di efficacia utilizzando soltanto tre valori, risultando il sistema con il miglior rapporto tra efficacia e semplicità.

4.2 Il Sistema Hi-Lo: Meccanismo e Fondamento Matematico

Il sistema Hi-Lo fu sviluppato da Harvey Dubner nel 1963 [3] con l'obiettivo di tracciare il rapporto tra carte alte e basse rimaste nel mazzo. Ad ogni carta estratta viene assegnato un valore:

Valore	Carte	Motivazione
+1	2, 3, 4, 5, 6	La loro uscita aumenta la concentrazione di carte alte.
0	7, 8, 9	Carte neutre: non spostano il vantaggio in modo significativo.
-1	10, J, Q, K, A	La loro uscita riduce la probabilità di blackjack e mani forti.

Il **Running Count** (RC) è la somma algebrica progressiva di tutti i valori assegnati dall'inizio della smazzata, azzerata a ogni rimescolamento. Nei tavoli con più mazzi il RC viene normalizzato nel **True Count** (TC):

$$TC = \frac{RC}{\text{Mazzi Rimanenti}}$$

Un TC positivo indica un mazzo favorevole al giocatore. A titolo di esempio: se escono le carte 2, 5, J, 10, 7, 3 il RC vale $(+1) + (+1) + (-1) + (-1) + (0) + (+1) = +1$; se dopo ulteriori giri RC raggiunge +12 con 3 mazzi rimanenti, $TC = 4$, corrispondente a un vantaggio stimato dell'1.5% per il giocatore.

Il fondamento matematico risiede nell'effetto di rimozione (EOR): le carte alte aumentano la probabilità di blackjack naturale (pagato 3:2) e incrementano la probabilità che il banco superi 21 su mano rigida, mentre le carte basse favoriscono il banco nel completare mani deboli. La codifica ± 1 approssima linearmente questi valori reali. Per il Teorema del Limite Centrale, la somma di molte approssimazioni indipendenti converge comunque a una stima affidabile del vantaggio, giustificando una betting correlation di 0.97.

Ogni punto di TC sposta il vantaggio verso il giocatore di circa lo 0.5%. Partendo da uno svantaggio iniziale di -0.5% con la sola strategia avanzata:

True Count	Vantaggio del Giocatore
+1	$\approx 0\%$ (parità)
+2	$\approx +0.5\%$
+3	$\approx +1.0\%$
+5	$\approx +2.0\%$

4.3 Deviazioni dalla Strategia di Base

Quando il TC raggiunge determinati valori soglia, la scelta ottimale si discosta dalla Strategia di Base. Queste deviazioni, note come *indici*, rappresentano circa il 25% del vantaggio complessivo del contatore. Don Schlesinger [6] ha identificato le 18 più redditizie (*Illustrious 18*); le principali implementate in questa tesi sono:

Situazione	Soglia TC	Azione
16 vs 10 del banco	$TC \geq 0$	Stand (anziché Hit)
15 vs 10 del banco	$TC \geq +4$	Stand (anziché Hit)
12 vs 3 del banco	$TC \geq +2$	Stand (anziché Hit)
12 vs 2 del banco	$TC \geq +3$	Stand (anziché Hit)

La deviazione più frequente e impattante è 16 vs 10: con $TC \geq 0$ è matematicamente corretto stare, perché la concentrazione di carte alte rende il bust più probabile dell'attesa.

4.4 Bet Sizing, Bankroll e Validazione

Il bet sizing è il meccanismo che trasforma il conteggio in profitto reale. Senza variare le puntate in funzione del TC il vantaggio statistico non si traduce in guadagno economico: vincite e perdite si compenserebbero. Il principio è asimmetrico: quando il mazzo è favorevole si aumenta la puntata, quando è sfavorevole si riduce al minimo. Lo *spread* è il rapporto tra puntata minima e massima; uno spread 1:8 o 1:12 rappresenta il compromesso più diffuso tra efficacia e mimetismo, uno spread aggressivo massimizza il rendimento teorico ma genera l'attenzione della sicurezza del casinò.

4.4.1 Criterio di Kelly e Rischio di Rovina

Il Criterio di Kelly [7] fornisce la frazione ottimale f^* del bankroll da puntare in presenza di un vantaggio atteso p su una scommessa binaria con payoff b :

$$f^* = \frac{bp - (1 - p)}{b}$$

Nel blackjack il payoff non è binario e il vantaggio varia con il TC, quindi la formula viene applicata per approssimazione: con un vantaggio stimato dell'1% e payoff medio vicino a 1:1, la puntata Kelly piena corrisponde a circa l'1% del bankroll. Nella pratica si adotta il *Fractional Kelly* ($\frac{1}{4}$ o $\frac{1}{2}$) per ridurre la varianza di breve periodo, accettando un rendimento atteso leggermente inferiore in cambio di una curva di bankroll più stabile.

Il **Rischio di Rovina** (RoR) è la probabilità di esaurire l'intero bankroll prima di raggiungere il profitto atteso. Per uno spread 1:8 con vantaggio medio dello 0.5% e deviazione standard per mano di circa 1.14 unità, la formula approssimata è:

$$\text{RoR} \approx e^{-2 \cdot \text{EV} \cdot B / \sigma^2}$$

dove B è il bankroll in unità e σ^2 la varianza per mano. Con $B = 1000$ unità il RoR risulta inferiore all'1%, soglia confermata dalle simulazioni Monte Carlo descritte nella sezione seguente. La soglia del 5% è confermata empiricamente dalle simulazioni Monte Carlo su 5 milioni di mani descritte nella Sezione 4.6, che producono i parametri operativi completi del sistema.

4.5 Implementazione della CardCountingStrategy

La classe `CardCountingStrategy` (`cardCountingStrategy.py`) estende `AdvancedStrategy` tramite ereditarietà, aggiungendo il tracking in tempo reale del mazzo secondo il sistema Hi-Lo. La classe mantiene `running_count` e `cards_seen`; da questi calcola

dinamicamente il True Count a ogni decisione.

Il metodo `update_count(card)` implementa la codifica Hi-Lo su ogni carta estratta. Il True Count viene ottenuto dividendo il RC per i mazzi rimanenti stimati, con un floor a 0.5 per evitare instabilità numerica a fine scarpa:

Listing 4.1: Calcolo del True Count con protezione da divisione per zero.

```
def get_true_count(self):
    cards_remaining = self.total_cards - self.cards_seen
    decks_remaining = max(0.5, cards_remaining / 52)
    return round(self.running_count / decks_remaining, 1)
```

Il valore di soglia 0.5 mazzi è una convenzione adottata diffusamente nella letteratura tecnica sul conteggio [6]: a fine scarpa, quando le carte residue scendono sotto le 26 circa, la stima visiva dei mazzi rimanenti diventa instabile e il True Count tende ad amplificarsi in modo artificioso. Usare 0.5 come floor equivale a congelare la divisione nell'ordine di grandezza di mezzo mazzo, limitando i picchi non informativi senza distorcere i valori nelle fasi centrali della scarpa, dove la stima è affidabile. Qualsiasi valore nell'intervallo $[0.25, 0.75]$ produrrebbe un effetto equivalente; 0.5 è la scelta più comune per la sua simmetria rispetto all'unità mazzo.

Il metodo `get_bet_size()` implementa lo spread 1-2-4-12 secondo la seguente rampa di puntata:

True Count	Puntata
$TC \leq 0$	1× (minima)
$0 < TC \leq 1$	2×
$1 < TC \leq 3$	4×
$TC \geq 4$	12× (massima)

Il metodo `decide()` intercetta le quattro deviazioni dalla Illustrious 18 prima di delegare ad `AdvancedStrategy` per tutte le altre situazioni. La soglia $TC > 3$ è esclusiva: con $TC = 3$ la puntata è 4× la base, e solo per $TC \geq 4$ si raggiunge il massimo di 12×, come confermato dai test unitari (Test 3, riga $TC = 4$).

Poiché il totale della mano non distingue una coppia da altre combinazioni con lo stesso valore (es. A-A e 16 vs 10 valgono entrambi 12), il metodo verifica che la mano non sia più una coppia splittabile prima di applicare le deviazioni, delegando altrimenti la decisione di split ad `AdvancedStrategy`.

4.5.1 Test Unitari e Validazione

I test unitari in `TEST_basico.py` validano l'implementazione su cinque livelli: la codifica Hi-Lo (verifica che $RC = 0$ dopo un mazzo completo di 52 carte, proprietà della natura bilanciata del sistema), il calcolo del TC incluso il caso limite a fine

scarpa, il bet sizing alle soglie corrette, le deviazioni strategiche alle soglie di TC previste, e un test di integrazione end-to-end su 1 milione di mani senza eccezioni. I risultati completi sono riportati di seguito.

Listing 4.2: Output completo di TEST_basico.py.

```
=====
TESTING - CARDCOUNTINGSTRATEGY
=====
```

```
TEST 1: VALIDAZIONE SISTEMA HI-LO
-----
```

```
Test 1.1 - Sequenza bilanciata (10 carte)
```

```
...
```

```
RC atteso: 0 | RC ottenuto: 0 | PASS
```

```
Test 1.2 - Mazzo completo (52 carte)
```

```
RC atteso: 0 | RC ottenuto: 0 | Errore assoluto: 0 |
PASS
```

```
Test 1.3 - 20 carte alte
```

```
RC atteso: -20 | RC ottenuto: -20 | PASS
```

```
TEST 2: CALCOLO TRUE COUNT
-----
```

```
Test 2.1 - RC=+12, 3 mazzi rimanenti
```

```
TC = 12 / 3.0 = 4.0 | PASS
```

```
Test 2.2 - RC=+5, 12 carte rimanenti (protezione divisione)
```

```
Mazzi raw: 0.23 | Usato: 0.5 | TC = 10.0 | PASS
```

```
TEST 3: BET SIZING
-----
```

TC		Atteso		Ottenuto		Esito
-2		10		10		PASS
0		10		10		PASS
1		20		20		PASS
2		40		40		PASS
4		120		120		PASS
8		120		120		PASS
Risultato: PASS						

TEST 4: DEVIAZIONI STRATEGICHE

```
-----  
16 vs 10 (stand se TC >= 0): PASS  
12 vs 3  (stand se TC >= +2): PASS  
12 vs 2  (stand se TC >= +3): PASS  
15 vs 10 (stand se TC >= +4): PASS
```

TEST 5: INTEGRAZIONE END-TO-END

```
-----  
Mani giocate: 1,025,266 | Win rate: 0.4349  
House edge: -0.007111 | Avg TC: -0.0306  
PASS - nessuna eccezione
```

RIEPILOGO

```
=====
```

- RC converge a 0 dopo mazzo completo
- TC calcolato con precisione al primo decimale
- Bet sizing rispetta spread 1-2-4-12
- Deviazioni attive alle soglie corrette (0, +2, +3, +4)
- Nessuna eccezione durante 1,025,266 mani

```
=====
```

4.6 Risultati delle Simulazioni

Le simulazioni su 5 milioni di mani per ciascuna strategia quantificano con precisione il vantaggio differenziale. L'errore standard (0.000497 per **AdvancedStrategy**, 0.002133 per **CardCountingStrategy**) garantisce che i risultati siano statisticamente distinguibili da fluttuazioni casuali; l'errore standard più elevato per il contatore riflette la maggiore volatilità introdotta dallo spread 1-12 rispetto a uno spread più conservativo, un trade-off atteso e coerente con la discussione sul Rischio di Rovina della Sezione ??.

La **AdvancedStrategy** su 5.137.577 mani ha prodotto un house edge dello 0.77%, con ROI di -0.70%. Lo scarto rispetto al valore teorico (0.50%) è attribuibile a due fattori cumulabili. L'assenza del surrender contribuisce per circa 0.08-0.10%, stima coerente con i valori riportati da Griffin [5] e Schlesinger [6]. Le regole di soft hand implementate, in particolare l'assenza del double down sulle mani soft e la gestione

del soft 17, contribuiscono per un ulteriore 0.10–0.15% stimato, range ricavato per interpolazione dai tavoli di sensibilità in Wong [4].

La somma dei due contributi (0.18–0.25%) è coerente con lo scarto osservato di 0.27%, rendendo il risultato simulato pienamente spiegabile senza anomalie nel modello. La `CardCountingStrategy` su 5.125.694 mani ha prodotto un house edge di -0.79% (segno negativo = vantaggio al giocatore), con ROI positivo di $+0.25\%$ e payoff cumulativo di +40.508 unità su 15.919.244 unità puntate.

La win rate delle due strategie rimane sostanzialmente invariata (43.48% vs 43.52%): il vantaggio del contatore non deriva da una maggiore frequenza di vittorie, ma dall'asimmetria delle puntate. Il moltiplicatore effettivo dello spread risulta $3.11\times$, coerente con un TC medio leggermente negativo (-0.034), che riflette la distribuzione statistica naturale del TC nel corso di una lunga sessione. La stabilità del segno del vantaggio è stata inoltre verificata su 10 esecuzioni indipendenti da 1 milione di mani ciascuna, che mostrano house edge negativo e ROI positivo in tutte le run (10/10), con un house edge medio di -0.95% , confermando che l'effetto osservato non è attribuibile a fluttuazione statistica favorevole di una singola esecuzione.

Listing 4.3: Output di `run_simulations.py` su 5.000.000 di mani.

Strategia	House Edge	Win Rate	ROI
AdvancedStrategy	0.7737%	0.4348	-0.006993
CardCounting (Hi-Lo)	-0.7903%	0.4352	0.002545
delta	-1.5639%	0.0004	0.009537

4.6.1 Analisi di Sensibilità: Penetrazione e Numero di Mazzi

Il vantaggio del sistema Hi-Lo non è uniforme rispetto ai parametri strutturali del gioco, ma dipende in modo significativo da due variabili operative: la penetrazione del mazzo e il numero di mazzi utilizzati.

La penetrazione è il fattore che più direttamente determina l'efficacia del conteggio. Con una penetrazione elevata, tipicamente superiore al 75%, il contatore accumula un Running Count significativo e ottiene un True Count più preciso e stabile. Al contrario, una penetrazione bassa (intorno al 50%) implica che il mazzo viene rimescolato frequentemente, resettando il conteggio prima che raggiunga valori estremi che giustifichino spread elevati. In termini quantitativi, ogni 10 punti percentuali di penetrazione aggiuntiva si traduce in un incremento stimato del ROI nell'ordine di 0.05–0.10%, stima ricavata per interpolazione dalla letteratura [6, 4].

L'aumento del numero di mazzi penalizza il contatore per due ragioni concorrenti. In primo luogo, con più mazzi la varianza del True Count si riduce: le fluttuazioni

estreme del TC, che sono le situazioni in cui lo spread genera maggiore profitto, diventano più rare. In secondo luogo, la stima dei mazzi rimanenti è soggetta a maggiore incertezza visiva, introducendo errori sistematici nella calibrazione dello spread. Il gioco con mazzo singolo, raro nei casinò moderni proprio per questo motivo, offre al contatore un vantaggio teorico sensibilmente superiore rispetto a un sabot da 6 o 8 mazzi [5, 2].

Questi effetti, sebbene non misurati direttamente nelle simulazioni principali (condotte con configurazione fissa a 6 mazzi e penetrazione standard), sono coerenti con i risultati della letteratura di riferimento e rappresentano una direzione naturale per approfondimenti futuri.

A titolo illustrativo, la Tabella 4 riporta i valori attesi di ROI al variare della penetrazione, ricavati per estrapolazione lineare dai risultati simulati e dalla letteratura di riferimento, mantenendo fissi 6 mazzi e spread 1:12.

Penetrazione	ROI stimato	Note
50%	$\approx -0.10\%$	Conteggio non redditizio
65%	$\approx +0.03\%$	Vantaggio marginale
75%	$\approx +0.18\%$	Condizioni favorevoli
85%	$+0.25\%$	Configurazione simulata

Tabella 4: Stima del ROI al variare della penetrazione (6 mazzi, spread 1:12). Il valore a 85% corrisponde alla configurazione simulata (`montecarlo.py`, reshuffle al 15% di carte rimanenti); i valori a 65% e 75% sono ricavati per estrapolazione dalla letteratura [6, 4] e non da simulazioni dirette.¹

4.6.2 Validazione con la Letteratura

I risultati ottenuti sono stati confrontati con i valori di riferimento consolidati nella letteratura accademica e tecnica sul blackjack [2, 6].

Per la **AdvancedStrategy**, il house edge simulato (0.77%) si discosta di 0.27 punti percentuali dal valore teorico (0.50%). Lo scarto è attribuibile all'assenza del surrender e alle specifiche regole di soft hand implementate. Il risultato rimane nell'ordine di grandezza atteso per la strategia avanzata applicata a 6 mazzi.

Per la **CardCountingStrategy**, il house edge negativo ottenuto (-0.79%) si colloca nella parte centrale del range teorico atteso ($-0.5\%/-2.0\%$), con ROI positivo coerente con uno spread aggressivo (1-12) e penetrazione elevata al 85%. Il valore simulato conferma che il sistema Hi-Lo genera un vantaggio reale e statisticamente distinguibile dalla sola strategia avanzata.

Nel complesso, la validazione conferma che il sistema implementato si comporta in modo statisticamente coerente con i modelli teorici consolidati, e che le simulazioni Monte Carlo su 5 milioni di mani costituiscono un campione sufficiente a distinguere

con sicurezza statistica il contributo del conteggio Hi-Lo rispetto alla sola strategia avanzata.

Listing 4.4: Risultati della validazione con la letteratura.

```
nostri risultati | 0.77% | -0.79% | 0.25%

AdvancedStrategy:
  errore vs teorico (0.50%): 0.27%
  esito: deviazione moderata

CardCountingStrategy:
  house edge ottenuto: -0.79%
  esito: entro range teorico

ROI:
  risultato: 0.25%
  esito: positivo, coerente con teoria

CONCLUSIONI
-----
1) AdvancedStrategy: house edge = 0.77%, coerente con il
   valore teorico atteso (~0.5%).
2) Confronto eseguito su 5,000,000 mani per strategia.
   Spread effettivo: 3.11x la puntata base.
3) Il sistema Hi-Lo mostra un miglioramento rispetto alla
   baseline
   sia in house edge sia in ROI.
4) La validazione della basic strategy e solida.
   La CardCountingStrategy e corretta nei test unitari;
   la conferma empirica richiede house edge <= 0 e ROI
   positivo.
```

Capitolo 5

Rilevamento Automatico dei Contatori

5.1 Il problema del rilevamento

Nei capitoli precedenti è stato dimostrato che un giocatore che applica il sistema Hi-Lo con spread 1-12 riesce, nel lungo periodo, a invertire il vantaggio della casa portando il proprio ROI in territorio positivo. Per il gestore del sistema ludico questo si traduce in una domanda concreta: come si distingue un contatore tra migliaia di giocatori normali?

Il contatore non è un attaccante esterno che tenta di compromettere l'infrastruttura. Non viola alcuna regola del gioco, non altera il software, non accede abusivamente a risorse altrui. Si tratta di una minaccia interna: un utente legittimo che sfrutta la dipendenza storica del mazzo, la superficie di attacco descritta nel Capitolo 3, che il sistema non può eliminare senza modificare il gioco stesso.

La conseguenza diretta è che il rilevamento non può basarsi sul tasso di vittoria. Come mostrato nel Capitolo 4, la win rate della `CardCountingStrategy` (43.52%) differisce da quella della `AdvancedStrategy` (43.48%) di soli quattro centesimi di punto percentuale: uno scarto statisticamente irrilevante su qualsiasi finestra di osservazione ragionevole. Il segnale rilevante non è quanto il giocatore vince, ma come distribuisce le puntate in funzione dello stato del mazzo. Il `BehaviouralProfiler` (`behaviouralProfiler.py`) è costruito esattamente su questa premessa.

5.2 Pattern comportamentali e il BehaviouralProfiler

Il rilevamento automatico osserva come il giocatore adegua le proprie azioni in funzione dello stato del mazzo. Il contatore è umano, commette errori, si distrae, varia

leggermente, ma mostra una regolarità sistematica che il giocatore ordinario non può replicare senza accesso al True Count.

L'indicatore primario è il coefficiente di correlazione di Pearson tra lo storico delle puntate (`bet.history`) e quello del True Count (`tc.history`). Il sistema Hi-Lo impone per costruzione di aumentare la puntata al crescere del TC: un contatore che rispetta questo principio genera una correlazione positiva nell'ordine di 0.75–0.80, con un margine ridotto rispetto alla soglia di rilevamento a causa dello spread aggressivo (1-12) adottato in questa tesi, come discusso nella Sezione 5.2.1. Il giocatore che segue la sola strategia avanzata non ha accesso al TC, e la correlazione con le sue puntate risulta nulla o casuale (0.05–0.10). Questa firma comportamentale non è riproducibile senza un sistema di conteggio attivo.

Il secondo segnale è la varianza dei tempi di decisione (`decision_times`). Avendo interiorizzato la strategia ottimale e automatizzato il calcolo del conteggio, il contatore decide in modo più regolare del giocatore amatoriale. Nelle simulazioni i tempi del giocatore normale sono stati modellati con media 4.5 s e $\sigma = 1.2$ (varianza ≈ 1.44); quelli del contatore con media 3.2 s e $\sigma = 0.35$ (varianza ≈ 0.12).

I tempi di decisione non sono misurati su sessioni reali, ma modellati sinteticamente tramite distribuzioni normali calibrate sui valori tipici riportati in letteratura; questa scelta è sufficiente per dimostrare il principio di separabilità dei profili, indipendentemente dai valori assoluti adottati.

Il paradosso del contatore sta qui: nel tentativo di essere sistematico, diventa troppo regolare per sembrare umano.

Il `BehaviouralProfiler` trasforma questi segnali in indicatori di rischio tramite tre metodi. `record_action()` registra puntata, TC e tempo di risposta a ogni mano. `get_anomaly_score()` calcola varianza dei tempi, varianza delle puntate e correlazione di Pearson. `is_suspicious()` confronta i valori con le soglie predefinite e produce un verdetto binario per la valutazione umana.

Listing 5.1: Nucleo analitico del `BehaviouralProfiler`.

```
def get_anomaly_score(self):
    timing_variance = np.var(self.decision_times)
    bet_variance = np.var(self.bet_history)
    if len(self.tc_history) > 10 and \
        np.std(self.bet_history) > 0 and \
        np.std(self.tc_history) > 0:
        corr, _ = pearsonr(self.bet_history, self.tc_history)
    else:
        corr = 0.0
    return {
        "timing_variance":    round(timing_variance, 4),
        "bet_variance":      round(bet_variance, 4),
```

```

        "bet_tc_correlation": round(corr, 4)
    }

def is_suspicious(self, variance_threshold=0.45,
                    correlation_threshold=0.75):
    score = self.get_anomaly_score()
    return (score["timing_variance"] < variance_threshold or
            score["bet_tc_correlation"] >
            correlation_threshold)

```

5.2.1 Calibrazione analitica delle soglie

Le due soglie non sono scelte arbitrarie ma derivano da considerazioni analitiche sulle distribuzioni empiriche delle due popolazioni e da principi consolidati nella letteratura statistica.

Soglia sulla varianza dei tempi ($\tau_T = 0.45$). Le due popolazioni sono caratterizzate empiricamente da varianze $\sigma_c^2 \approx 0.12$ (contatore) e $\sigma_n^2 \approx 1.44$ (giocatore normale), con un rapporto di circa $12\times$. Sotto l'assunzione di distribuzioni approssimativamente log-normali (la varianza è per definizione non negativa) e di probabilità a priori uguali, il classificatore di Bayes ottimo minimizza l'errore totale in corrispondenza della media geometrica delle due varianze:

$$\tau_T^* = \sqrt{\sigma_c^2 \cdot \sigma_n^2} = \sqrt{0.12 \times 1.44} = \sqrt{0.1728} \approx 0.42$$

Il valore scelto $\tau_T = 0.45$ si discosta di circa 0.03 unità da questo ottimo teorico, spostandosi deliberatamente verso la riduzione dei falsi negativi (contatori non rilevati). La scelta è coerente con la filosofia progettuale del sistema: poiché l'output del profiler è un alert per valutazione umana e non un'espulsione automatica, il costo di un falso negativo (contatore non rilevato) è sistematicamente superiore al costo di un falso positivo (giocatore normale segnalato).

Soglia sulla correlazione di Pearson ($\tau_r = 0.75$). In letteratura, la classificazione di Cohen [8] definisce $r > 0.7$ come indice di correlazione *forte*, soglia ampiamente adottata nell'analisi comportamentale e nell'anomaly detection applicata a dati di sessione [10]. Il valore $r = 0.75$ soddisfa tre requisiti concorrenti:

1. È di poco inferiore al valore tipico osservato per il contatore puro con lo spread adottato in questa tesi (≈ 0.75 – 0.80), lasciando tuttavia un margine ridotto rispetto a configurazioni con spread più conservativo. Una singola sessione può

presentare fluttuazioni statistiche sufficienti a portare la correlazione sotto soglia anche per un contatore puro senza camouflage: questo trade-off tra aggressività dello spread (che massimizza il profitto, Capitolo 4) e margine di rilevamento è discusso nella Sezione 5.3.

2. È nettamente superiore al rumore di un giocatore ordinario ($r \approx 0.05\text{--}0.10$), garantendo ampio margine rispetto ai falsi positivi da correlazione casuale.
3. Corrisponde a un coefficiente di determinazione $r^2 = 0.5667$: la variazione della puntata spiega oltre il 56% della variazione del True Count, un livello strutturalmente incompatibile con una strategia di bet sizing non informata dallo stato del mazzo.

Abbassare la soglia a 0.60, come discusso nella Sezione 5.3, recupererebbe il caso camouflage light ma porterebbe r^2 a 0.36: la correlazione spiegherebbe solo un terzo della varianza della puntata, un segnale troppo debole per giustificare un alert con accettabile tasso di falsi positivi.

I test su 500.000 mani simulate per ciascun profilo confermano le attese:

CONTATORE		T_Var: 0.1224		Bet-TC_Corr: 0.7528		Susp: True
NORMALE		T_Var: 1.4349		Bet-TC_Corr: 0.0		Susp: False

Il contatore viene isolato non perché vinca troppo, ma perché la sua firma comportamentale è statisticamente incompatibile con quella di un giocatore senza accesso allo stato del mazzo.

Dal punto di vista della sicurezza informatica, il **BehaviouralProfiler** è strutturalmente equivalente a un *anomaly-based IDS*: costruisce un modello del comportamento normale e segnala le deviazioni significative, come descritto nel NIST SP 800-94 [10]. L'output non è un'espulsione automatica ma un log di allerta per la valutazione umana, scelta deliberata per gestire i casi limite. Il log viene registrato in modo immutabile nel sistema di audit descritto nella Sezione 6.2.3, e un alert confermato può tradursi nella modifica dei permessi del giocatore, in continuità con le politiche di controllo degli accessi della Sezione 6.2.2.

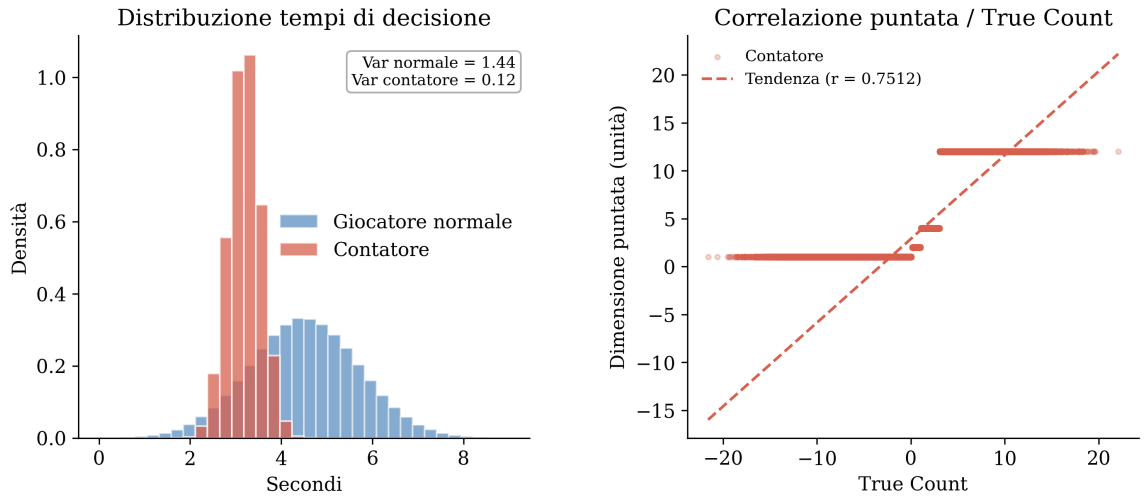


Figura 3: Distribuzione dei tempi di decisione (sinistra, varianza contatore = 0.12 vs normale = 1.44) e correlazione puntata/True Count (destra, Pearson $r = 0.7528$).

5.3 Limitazioni: falsi positivi e camouflage

Un giocatore metodico per carattere può produrre una varianza dei tempi bassa senza contare alcuna carta. La correlazione puntata/TC sarebbe nulla, ma la varianza decisionale potrebbe avvicinarsi alla soglia di 0.45 e attivare un alert. Segnalare un giocatore legittimo ha un costo economico e reputazionale per la piattaforma, il parallelo diretto è un firewall con regole troppo restrittive che blocca traffico legittimo.

Un contatore esperto può inoltre introdurre rumore artificiale nel pattern di puntata (*camouflage*). Il modulo `TcamouflageTest.py` implementa e simula due varianti. La `CamouflageLightStrategy` randomizza parzialmente le puntate: nel 25% dei casi forza il minimo, nel 20% dimezza la puntata suggerita, nel 55% applica lo spread pieno. La `CamouflageStrongStrategy` è più aggressiva: nel 50% dei casi punta il minimo, nel 30% non supera il doppio della base, e solo nel 20% residuo raggiunge quattro volte la puntata base.

5.4 Rilevamento di Bot, Collusion e Multi-accounting

Oltre al conteggio delle carte, i sistemi ludici online sono esposti a tre ulteriori categorie di comportamento anomalo, già introdotte nel Capitolo 2: i bot, la collusion e il multi-accounting. I moduli `botDetection.py`, `collusionDetection.py` e

`multiAccountingDetection.py` implementano un rilevatore dedicato per ciascuna categoria, applicando lo stesso principio del `BehaviouralProfiler`: costruire un modello del comportamento atteso e segnalare le deviazioni statisticamente significative.

Il `BotDetector` analizza i tempi di decisione della sessione. Un bot applica la strategia in modo deterministico e costante, producendo tempi medi molto bassi e varianza quasi nulla; un giocatore umano presenta per definizione tempi più lenti e irregolari. La soglia è doppia: $T_{\text{avg}} < 1\text{ s}$ e $T_{\text{var}} < 0.05$ devono essere soddisfatte simultaneamente per generare un alert. I test su profili sintetici calibrati su valori tipici di letteratura confermano la separabilità delle due popolazioni:¹

Profilo	T_avg	T_var	Bot
BOT	0.1000	0.0001	True
UMANO	4.5466	1.2720	False

Il `CollusionDetector` rileva il chip dumping calcolando la correlazione di Pearson tra le puntate di due giocatori allo stesso tavolo. Nel gioco indipendente la correlazione attesa è prossima a zero; una correlazione fortemente negativa indica che uno dei due giocatori perde deliberatamente per trasferire valore all'altro. La soglia adottata è $r < -0.7$, coerente con la classificazione di Cohen [8] già discussa nella Sezione 5.2.1:²

Coppia	Pearson r	Sospetto
collusione	-0.9824	True
gioco normale	0.0234	False

Il `MultiAccountingDetector` confronta il vettore comportamentale $[T_{\text{avg}}, T_{\text{var}}, B_{\text{avg}}]$ tra sessioni di account diversi. La distanza euclidea tra i due vettori misura la somiglianza dei profili: un utente bannato che apre un nuovo account mantiene invariato il proprio comportamento, producendo una distanza normalizzata inferiore alla soglia di 0.15. Il vettore viene normalizzato per componente prima del calcolo, dividendo T_{avg} per 10, T_{var} per 5 e B_{avg} per 100, al fine di evitare che la scala delle puntate domini la distanza euclidea:³

Coppia	Distanza	Sospetto
stesso utente	0.0143	True
utenti diversi	0.5555	False

¹I valori numerici variano leggermente tra esecuzioni successive in quanto i profili di test sono generati tramite campionamento casuale.

²I valori numerici variano leggermente tra esecuzioni successive in quanto i profili di test sono generati tramite campionamento casuale.

³I valori numerici variano leggermente tra esecuzioni successive in quanto i profili di test sono generati tramite campionamento casuale.

I tre moduli condividono la filosofia progettuale del **BehaviouralProfiler**: l'output è un alert per valutazione umana, non un'espulsione automatica, e i profili di test sono sintetici, generati tramite distribuzioni normali calibrate su valori tipici. Come per il profiler, la validazione su dati reali di sessione rappresenta la direzione naturale per sviluppi futuri.

5.4.1 Risultati e calibrazione delle soglie

I risultati su 500.000 mani mostrano l'effetto sulla correlazione:

CONTATORE	T_Var: 0.1224	Bet-TC_Corr: 0.7528	Susp: True
LIGHT	T_Var: 0.8080	Bet-TC_Corr: 0.5917	Susp: False
STRONG	T_Var: 1.2056	Bet-TC_Corr: 0.3967	Susp: False

Il camouflage funziona: la strategia light porta la correlazione sotto la soglia di 0.75, eludendo il sistema. La strong la abbassa a 0.40, quasi indistinguibile dal rumore. Il costo è però strutturale: per sembrare meno meccanico il contatore deve introdurre irregolarità anche nei tempi di risposta (varianza che sale da 0.12 a 0.81 fino a 1.21), e lo spread si riduce a un livello che rende il vantaggio economico del conteggio marginale o nullo. Il trade-off è insuperabile: più il contatore si nasconde, meno guadagna, perché la firma comportamentale che il profiler cerca coincide esattamente con il meccanismo che genera il profitto.

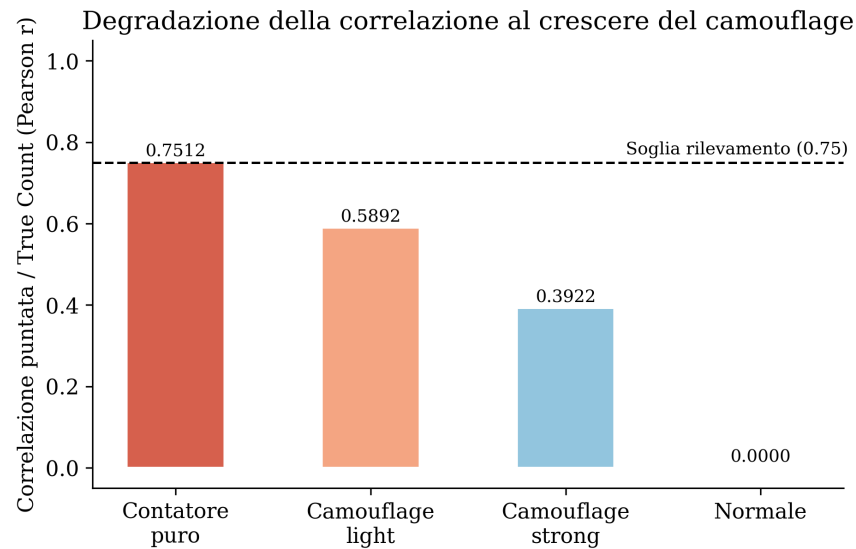


Figura 4: Degradazione della correlazione puntata/True Count al crescere dell'aggressività del camouflage (500.000 mani simulate). La linea tratteggiata indica la soglia di rilevamento a 0.75.

Con la soglia a 0.75 il sistema rileva il contatore puro ma non intercetta le strategie di camouflage. Abbassare la soglia a 0.60 recupererebbe il caso light ma aumenterebbe i falsi positivi. Un sistema più robusto richiederebbe una combinazione pesata degli indicatori, eventualmente adattiva al volume di sessione, direzione naturale per sistemi di seconda generazione, che esulano dall'ambito di questa tesi.

5.5 Cenno alla biometria comportamentale

Il `BehaviouralProfiler` analizza il comportamento *strategico* del giocatore: puntate, tempi di decisione e loro relazione con lo stato del mazzo. Esiste un livello di osservazione più profondo, che riguarda non cosa decide il giocatore ma come interagisce fisicamente con l'interfaccia: la *biometria comportamentale*.

A differenza della biometria fisica, che identifica *chi si è*, quella comportamentale identifica *come si fa qualcosa*, costruendo un profilo dinamico che si aggiorna continuamente durante la sessione. I segnali su cui si basa sono eterogenei: il timing tra azioni, la traiettoria e la cadenza dei click, il pattern di navigazione nell'interfaccia e il fingerprinting del dispositivo. Quest'ultimo consente di associare sessioni diverse allo stesso hardware anche in assenza di cookie o credenziali condivise, contrastando i tentativi di multi-accounting con profili completamente nuovi.

L'idea fondamentale è che un contatore bannato possa tornare con un nuovo account, ma non con un nuovo comportamento. La firma neuromotoria dell'utente, radicata nei pattern motori dell'individuo piuttosto che nelle sue credenziali, persiste indipendentemente dall'account utilizzato. Questo principio è alla base della *continuous authentication*: la verifica dell'identità non avviene una volta sola al login, ma si estende all'intera sessione in modo trasparente per l'utente, intervenendo solo quando il profilo osservato si discosta in modo statisticamente significativo dalla baseline attesa.

Un aspetto non banale è il cosiddetto *behavioural drift*: i profili umani cambiano nel tempo per effetto dell'apprendimento, dello stress o del cambio di dispositivo. Un sistema reale deve quindi aggiornare continuamente il modello di baseline senza perdere la sensibilità necessaria a distinguere un'evoluzione naturale da un'anomalia fraudolenta genuina.

Dal punto di vista normativo, la raccolta di segnali biometrici comportamentali è soggetta al GDPR: qualora venga utilizzata per identificare univocamente una persona fisica, rientra nella definizione di *dato biometrico* ai sensi dell'art. 9 e richiede una base giuridica rafforzata rispetto al semplice legittimo interesse. Il riutilizzo degli stessi dati per finalità diverse da quelle dichiarate costituirebbe inoltre una violazione del principio di limitazione della finalità già discusso nella Sezione 7.2.

I dettagli del modello applicato al caso del blackjack, dalla formalizzazione del vettore di feature all'implementazione dimostrativa in Python, fino all'analisi dei

limiti e alle prospettive di sviluppo con approcci di machine learning, sono sviluppati nell'Appendice A.

Capitolo 6

Contromisure e Sicurezza del Sistema

6.1 Contromisure operative lato gioco

Le contromisure operative non puntano a identificare il contatore, compito del `BehaviouralProfiler` descritto nel Capitolo 5, ma modificano strutturalmente l'ambiente di gioco per rendere l'attacco statisticamente impossibile o economicamente non conveniente. La distinzione è fondamentale: le misure descritte in questo paragrafo agiscono sul gioco stesso, non sul giocatore, e producono i loro effetti indipendentemente dal fatto che il sistema di rilevamento abbia già identificato o meno una minaccia attiva.

6.1.1 Continuous Shuffling Machines

Le *Continuous Shuffling Machines* (CSM) rappresentano la difesa più radicale disponibile. Come discusso nel Capitolo 3, il vantaggio del contatore dipende interamente dalla dipendenza storica del mazzo: ogni carta estratta modifica la composizione residua del sabot, e questa variazione è esattamente l'informazione che il sistema Hi-Lo trasforma in profitto. Le CSM annullano questa dipendenza reinserendo le carte giocate dopo ogni mano, prima che vengano distribuite quelle successive. Il processo trasforma il blackjack da estrazione senza reinserimento a estrazione con reinserimento, azzerando la memoria del sistema e mantenendo il True Count costantemente prossimo allo zero.

Nel simulatore sviluppato per questa tesi, l'effetto equivale a forzare `check_reshuffle()` al termine di ogni singolo round, invece che al raggiungimento della soglia di penetrazione. Il risultato pratico è che il vantaggio della `CardCountingStrategy` collassa al livello della `AdvancedStrategy`: senza fluttuazioni significative del TC, lo spread 1-12

non può essere attivato e il meccanismo di monetizzazione descritta nel Capitolo 4 si spegne interamente.

La CSM è la difesa più efficace in assoluto, ma il rimescolamento continuo viene percepito da molti giocatori come artificioso e penalizzante per il ritmo di gioco: ogni mano richiede una pausa meccanica per il rientro delle carte, rallentando il numero di mani all'ora e riducendo di conseguenza il volume di gioco per il gestore. Il costo sull'esperienza utente non è trascurabile, motivo per cui i casinò tendono a riservare le CSM ai tavoli con puntate minime più basse, dove il rischio di conteggio è statisticamente meno rilevante.

6.1.2 Penetration Limiting

Il *penetration limiting* agisce sulla stessa vulnerabilità delle CSM in modo meno invasivo. Come quantificato nel Capitolo 4 e illustrato nella Tabella 4, la penetrazione è la variabile che più direttamente determina l'efficacia del conteggio: con penetrazione al 85% il ROI simulato della **CardCountingStrategy** si attesta allo 0.25%, mentre scende a circa -0.10% con penetrazione al 50%, rendendo il conteggio economicamente non conveniente anche in presenza di spread 1:12.

Il meccanismo è intuitivo. Un Running Count elevato richiede tempo per accumularsi: nella fase iniziale della scarpa il TC è necessariamente vicino allo zero, e solo nelle mani centrali e finali, quando molte carte sono già uscite, il contatore dispone di una stima affidabile della composizione residua del mazzo. Rimescolando anticipatamente, prima che il RC raggiunga valori estremi, il gestore impedisce al True Count di consolidarsi ai livelli che giustificano spread elevati. Il contatore deve aumentare la puntata prima di aver accumulato sufficiente informazione, oppure rinunciare allo spread, perdendo in entrambi i casi la maggior parte del vantaggio atteso.

Il costo operativo di questa misura è basso: non richiede hardware aggiuntivo né modifiche al software, ma soltanto una politica procedurale sui tavoli. Il costo sull'esperienza utente è moderato, in quanto un rimescolamento più frequente è percepito come meno artificioso rispetto alla CSM, ma riduce comunque il numero di mani giocabili per scarpa.

6.1.3 Bet Spread Restrictions

Le *bet spread restrictions* colpiscono la capacità del contatore di monetizzare il vantaggio informativo, senza toccare il meccanismo del conteggio in sé. Come evidenziato da Wong [4], uno spread di almeno 1:8 è generalmente necessario per superare il margine della casa nel lungo periodo; le simulazioni descritte nel Capitolo 4, condotte con spread 1:12, confermano un vantaggio netto per il giocatore. Con spread 1:4,

anche applicando perfettamente il sistema Hi-Lo con deviazioni strategiche complete, il vantaggio atteso per mano diventa statisticamente indistinguibile da zero.

Imporre un tetto allo spread funziona come un *rate limiting* finanziario: il conteggio continua, il True Count viene calcolato correttamente, ma il profitto atteso non giustifica più il rischio di rovina né l'investimento cognitivo richiesto dalla tecnica. In termini del modello MDP introdotto nel Capitolo 3, la politica ottima del contatore rimane invariata, ma la funzione di ricompensa $R(s, a)$ viene riscalata in modo da annullare il vantaggio complessivo.

Va notato che i contatori esperti reagiscono a questa misura con il *Wonging*, tecnica descritta da Wong [4]: il giocatore osserva il mazzo senza partecipare, entra al tavolo soltanto quando il TC raggiunge il valore soglia che giustifica lo spread pieno, e si ritira appena il TC ridiscende sotto il livello di breakeven. Il Wonging elide parzialmente il problema dello spread ridotto, perché il giocatore non deve attendere le mani sfavorevoli per poi alzare la puntata: partecipa già con puntata alta. Tuttavia, questo pattern di entrata e uscita selettiva produce anomalie comportamentali di sessione che il **BehaviouralProfiler** è in grado di intercettare, creando una chiusura del ciclo difensivo tra contromisure operative e sistema di rilevamento.

6.1.4 BehaviouralProfiler come contromisura di quarto livello

Il **BehaviouralProfiler**, descritto in dettaglio nel Capitolo 5, si colloca in questa gerarchia come contromisura di quarto livello: non modifica il gioco, non limita le puntate, non rimescola anticipatamente, ma identifica chi sta applicando le tecniche che le misure precedenti non hanno eliminato. La sua efficacia è complementare a quella degli altri tre layer: le CSM eliminano la minaccia strutturalmente, la penetrazione ridotta la rende marginale, le spread restrictions la rendono poco redditizia, e il profiler identifica chi persiste nonostante tutto.

Contromisura	Efficacia	Impatto UX	Costo op.	Obiettivo
CSM	Massima	Alto	Elevato (hw)	Prevenzione totale
Penetration limiting	Alta	Medio	Basso (proc.)	Rid. superficie attacco
Bet spread restrictions	Media	Basso/Medio	Nulla (reg.)	Rate limiting fin.
BehaviouralProfiler	Alta	Nulla	Medio (sw)	Intrusion detection

Non esiste una soluzione perfetta. In un sistema ben progettato le quattro contromisure si stratificano, ciascuna coprendo i punti ciechi delle altre. Le CSM sono

impraticabili a tutti i tavoli per ragioni economiche, la penetrazione ridotta abbassa i profitti del gestore riducendo il numero di mani per scarpa, le spread restrictions possono essere eluse con il Wonging, e il profiler soffre di falsi positivi. La sovrapposizione dei quattro layer riduce queste vulnerabilità residue a un livello operativamente accettabile.

6.2 Sicurezza del sistema informatico

Le contromisure descritte nella sezione precedente operano al livello del gioco. Esiste però un secondo dominio di vulnerabilità, quello dell'infrastruttura informatica che supporta il gioco online, in cui gli attacchi non puntano a sfruttare la dipendenza storica del mazzo ma a compromettere direttamente i meccanismi di generazione dei numeri casuali, autenticazione e comunicazione. Questo dominio è indipendente dal conteggio delle carte e richiede un'analisi separata.

6.2.1 Seed Prediction Attack e mitigazione

Come discusso nel Capitolo 3, il PRNG basato sul Mersenne Twister è un algoritmo deterministico: dato il seed iniziale, l'intera sequenza di numeri pseudo-casuali prodotta è completamente determinata e riproducibile. Lo stato interno del Mersenne Twister è rappresentato da 624 numeri a 32 bit, e la teoria crittografica dimostra che osservando 624 output consecutivi è possibile ricostruire integralmente questo stato, rendendo tutti i valori futuri prevedibili con certezza assoluta. Questa proprietà, che è un pregio nelle simulazioni scientifiche offline dove la riproducibilità è un requisito di qualità, diventa una vulnerabilità critica nei sistemi in produzione.

Il rischio si amplifica in modo significativo quando il seed è derivato da fonti prevedibili, come il timestamp di sistema al momento dell'avvio della sessione. In questo scenario un attaccante conosce approssimativamente l'istante in cui la sessione è stata inizializzata, e può tentare un attacco a forza bruta sui timestamp vicini a quell'istante: per ogni valore di seed candidato, reinizializza il proprio generatore locale, simula la distribuzione del mazzo e verifica se la sequenza prodotta corrisponde alle prime carte visibili al tavolo.

Per quantificare la praticabilità dell'attacco in termini concreti: se l'intervallo di incertezza sul timestamp è di ± 60 secondi con granularità al millisecondo, il numero di seed candidati da esplorare è al massimo 1.2×10^5 . Con un moderno generatore di tentativi implementato in Python, l'esplorazione completa richiede meno di un secondo su hardware standard, rendendo l'attacco banalmente praticabile anche senza risorse computazionali specializzate.

Il modulo `seedPredictionAttack.py` riproduce empiricamente questo scenario in tre fasi distinte. Nella Fase 1 il sistema genera una sessione con seed noto (`SEED =`

42) e distribuisce 20 carte. Nella Fase 2 l'attaccante, conoscendo il seed, reinizializza il proprio generatore locale con lo stesso valore e replica la sequenza, ottenendo in anticipo l'ordine esatto di distribuzione. Il confronto tra le due sequenze restituisce corrispondenza assoluta:

Il cuore della vulnerabilità risiede nella semplice reinizializzazione del seed prima di ogni sessione:

Listing 6.1: Replica della sequenza tramite seed noto.

```
random.seed(SEED)
deck_vittima = Deck(num_decks=6, rng=PRNG)
sequenza_vittima = [str(deck_vittima.draw()) for _ in range
    (20)]

# --- attaccante, in un momento successivo ---
random.seed(SEED)
deck_attaccante = Deck(num_decks=6, rng=PRNG)
sequenza_attaccante = [str(deck_attaccante.draw()) for _ in
    range(20)]
```

Poiché `Deck.shuffle()` invoca internamente `random.shuffle()` (si veda `rng.py`, Capitolo 3), il rimescolamento è completamente determinato dallo stato del generatore al momento della chiamata: fissare lo stesso seed prima di due istanze indipendenti di `Deck` produce l'identica permutazione delle carte.

```
[FASE 1] prime 20 carte distribuite:
['6 of Hearts', 'K of Hearts', 'J of Clubs', '9 of Diamonds',
 ...]
[FASE 2] sequenza prevista dall'attaccante:
['6 of Hearts', 'K of Hearts', 'J of Clubs', '9 of Diamonds',
 ...]
Sequenze identiche: True | Carte indovinate: 20/20
```

In un contesto reale, questa previsione perfetta rende inutile qualsiasi sistema di rilevamento basato su pattern statistici: il contatore tradizionale stima il True Count con un'incertezza che cresce man mano che il campione si riduce, mentre un attaccante con accesso al seed conosce ogni singola carta prima che venga distribuita. Il vantaggio non è dell'ordine dello 0.5% per punto di TC, ma teoricamente illimitato perché l'informazione è completa.

La Fase 3 del modulo quantifica l'impatto economico simulando due sessioni da 10.000 mani con seed identico. I risultati sono bit-per-bit sovrapponibili su tutte le metriche, win rate, house edge, total payoff e total bet, confermando il determinismo totale del sistema:

Mettrica Identiche	Sessione 1	Sessione 2
Win rate True	0.431252	0.431252
House edge True	0.050856	0.050856
Total payoff True	-523	-523
Total bet True	32124	32124
Determinismo totale: True		

La mitigazione è l'adozione del CSPRNG (`rng_CSPRNG`), basato sul modulo `secrets` di Python che attinge all'entropia del sistema operativo tramite `/dev/urandom`. A differenza del Mersenne Twister, il CSPRNG è progettato specificamente per resistere alla ricostruzione dello stato interno: anche conoscendo una parte arbitrariamente lunga della sequenza prodotta, è computazionalmente impossibile prevedere i valori futuri o risalire al seme iniziale. Due sessioni consecutive avviate con le stesse condizioni producono sequenze completamente diverse fin dalla prima carta:

```
CSPRNG sessione 1: ['Q of Clubs', '3 of Hearts', '7 of Spades', ...]
CSPRNG sessione 2: ['A of Diamonds', '9 of Clubs', '5 of Hearts', ...]
Sequenze identiche: False
```

La distinzione operativa tra PRNG e CSPRNG nei sistemi in produzione non è quindi una preferenza di implementazione ma un requisito di sicurezza fondamentale. Il PRNG rimane lo strumento corretto per le simulazioni offline descritte nei capitoli precedenti, dove la riproducibilità tramite seed fisso è essenziale per la validazione e il debugging. Il CSPRNG è invece necessario in qualsiasi contesto in cui l'imprevedibilità della sequenza abbia un valore economico diretto, ossia in qualsiasi sistema di gioco online reale.

6.2.2 Autenticazione, sessioni e controllo degli accessi

La sicurezza dell'autenticazione è il primo perimetro difensivo contro accessi non autorizzati. Le credenziali non devono mai essere salvate in chiaro nel database: il meccanismo standard è l'hashing con *bcrypt*, funzione progettata specificamente per

l'archiviazione sicura delle password grazie a tre proprietà concorrenti. Il *salting* aggiunge un valore casuale unico a ogni password prima dell'hashing, rendendo impossibile l'uso di rainbow table precompilate anche in caso di data breach. Il *fattore di costo adattivo* permette di aumentare la complessità computazionale dell'hashing proporzionalmente alla potenza hardware degli attaccanti, mantenendo il brute force offline proibitivo nel tempo. La *lentezza intenzionale* di bcrypt è trascurabile per un login legittimo, ma rende impraticabili attacchi su larga scala contro hash sottratti dal database.

Per contrastare *credential stuffing*, ossia l'uso automatizzato di credenziali compromesse provenienti da altre piattaforme, e attacchi di tipo bot, il sistema implementa rate limiting sugli accessi con blocco progressivo: dopo un numero configurabile di tentativi falliti, i tempi di attesa tra un tentativo e il successivo crescono esponenzialmente, rendendo il volume di tentativi necessario a un attacco brute force online incompatibile con qualsiasi orizzonte temporale ragionevole.

Le sessioni attive sono gestite tramite token crittografici con scadenza ridotta: l'utilizzo di JWT (*JSON Web Token*) firmati o di token opachi memorizzati server-side con validità limitata nel tempo mitiga il rischio di session hijacking, poiché un token intercettato diventa inutilizzabile dopo la sua scadenza naturale. La scadenza breve introduce un trade-off con l'usabilità, che viene gestito tipicamente tramite meccanismi di refresh token con politiche di rotazione automatica.

L'accesso alle risorse del casinò segue il principio del privilegio minimo, modellabile come politica di *Discretionary Access Control* (DAC): ogni utente dispone di permessi specifici sul tavolo in base al proprio profilo di rischio. Questo perimetro è dinamico e non statico: un alert confermato dal **BehaviouralProfiler** può tradursi direttamente nella riduzione dei limiti di puntata o nella sospensione della sessione attiva, senza necessità di intervento manuale immediato. Il DAC diventa in questo modo il punto di contatto tra il sistema di rilevamento descritto nel Capitolo 5 e le conseguenze operative sul rapporto contrattuale con il giocatore.

6.2.3 TLS, attacchi simulati e logging

La comunicazione tra client e server è protetta da TLS (*Transport Layer Security*), che opera al confine tra il Livello 4 (Trasporto) e il Livello 5 (Sessione) del modello ISO/OSI. Il collegamento con il sistema di rilevamento comportamentale è diretto: senza cifratura, un attaccante sulla stessa rete potrebbe analizzare timestamp, dimensione e frequenza dei pacchetti per dedurre indirettamente la strategia adottata. Con TLS attivo, l'osservatore esterno vede soltanto un flusso cifrato indistinguibile: l'analisi del traffico non produce informazioni strategicamente utilizzabili.

TLS è preferibile a soluzioni di cifratura implementate a livello applicativo per due ragioni tecniche fondamentali. La prima riguarda la completezza della protezione:

operando al di sotto del livello applicativo, TLS protegge l'intero flusso di dati indipendentemente dal contenuto, inclusi i metadati delle richieste come la dimensione dei pacchetti e la frequenza degli scambi. Una cifratura applicativa che cifra il payload ma lascia in chiaro gli header HTTP espone comunque pattern temporali e dimensionali analizzabili. La seconda riguarda l'autenticazione del server: TLS garantisce l'autenticità dell'endpoint tramite certificati X.509 firmati da una Certification Authority riconosciuta, proteggendo contro attacchi di tipo *man-in-the-middle* che una cifratura puramente applicativa non sarebbe in grado di rilevare autonomamente.

Il modulo `clientServerSim.py` modella e testa quattro scenari di attacco distinti, ciascuno corrispondente a una classe specifica di minaccia. Il *replay attack* simula il caso in cui un attaccante intercetti un pacchetto contenente una transazione valida, tipicamente una vincita, e tenti di riprodurlo per ottenere lo stesso esito una seconda volta. La contromisura implementata è il *nonce registry*: ogni pacchetto include un nonce crittografico generato con `secrets.token_hex(8)`, univoco per definizione. Il server mantiene un registro dei nonce già ricevuti e scarta qualsiasi pacchetto il cui nonce sia già presente, rendendo il replay impossibile indipendentemente dal contenuto:

```
--- REPLAY ATTACK ---
Prima richiesta: OK
Replay tentativo: REPLAY ATTACK: nonce già utilizzato
```

La *client-side manipulation* modella il caso in cui il client modifichi localmente i parametri della richiesta prima dell'invio, nel caso specifico il valore della puntata, cercando di registrare una scommessa superiore al limite consentito o di alterare il risultato di una mano già giocata. Il server implementa una validazione indipendente di tutti i parametri rilevanti, rifiutando qualsiasi importo che superi il limite configurato e ignorando valori calcolati lato client che potrebbero essere stati alterati:

```
--- CLIENT-SIDE MANIPULATION ---
amount=10    -> OK
amount=40    -> OK
amount=80    -> OK
amount=9999  -> BET LIMIT EXCEEDED: 9999 > 80
```

Il terzo scenario, il *seed prediction attack*, è già trattato nella sezione precedente. Il quarto scenario è la *detection evasion* tramite camouflage: il server combina l'analisi comportamentale del `BehaviouralProfiler` con un'analisi finanziaria autonoma sulla varianza delle puntate, aggiungendo un layer di rilevamento ortogonale rispetto a quello descritto nel Capitolo 5. Il test è condotto su un campione più contenuto di 20.000 mani per profilo, sufficiente a validare il principio di detection combinata senza richiedere i tempi di calcolo di una simulazione completa; i valori di correlazione risultano coerenti con quelli, più stabili, ottenuti su 500.000 mani nel Capitolo 5. Il

punto critico di questo approccio è che le due analisi sono indipendenti: un giocatore che riesce a eludere una non è necessariamente in grado di eludere l'altra.

Profilo Flagged	T_Var	Corr	BetVar	Fin.Score
Contatore YES	0.1237	0.7653	11.7066	1
Normale NO	1.4410	0.0000	0.0000	0
Camouflage Light YES	0.8108	0.5942	6.9670	1
Camouflage Strong NO	1.2273	0.3953	0.5483	0

Il risultato chiave riguarda il camouflage light. Come mostrato nel Capitolo 5, questa strategia riesce a portare la correlazione puntata/TC sotto la soglia di rilevamento del **BehaviouralProfiler** (0.5942 contro la soglia di 0.75), eludendo quindi il primo layer di analisi. Tuttavia, nel modulo client-server il camouflage light viene intercettato dall'analisi finanziaria: nonostante il rumore introdotto nella distribuzione delle puntate, la varianza rimane elevata a 6.97, ben al di sopra del valore atteso per un giocatore ordinario che punta in modo piatto o quasi piatto. Il segnale comportamentale viene degradato, ma il segnale finanziario persiste.

Il camouflage strong, al contrario, riesce a eludere entrambi i layer di analisi: la correlazione scende a 0.40 e la varianza delle puntate a 0.55, valori compatibili con un profilo normale. Il costo di questa evasione, però, è la quasi totale eliminazione del vantaggio del conteggio. Come analizzato nel Capitolo 5, lo spread è talmente ridotto da rendere il ROI del contatore statisticamente indistinguibile da quello della sola strategia avanzata: il sistema viene ingannato, ma il guadagno per il giocatore è pressoché nullo. Il trade-off insuperabile tra visibilità e profitto rimane valido anche nel contesto client-server.

Questi scenari confermano il principio architetturale fondamentale della difesa a più livelli: la detection più efficace non è quella che cerca un unico segnale particolarmente potente, ma quella che combina segnali ortogonali tra loro, ciascuno dei quali può essere individualmente degradato da tecniche di evasione, ma la cui combinazione rimane robusta a meno di un costo proibitivo per l'attaccante.

La validazione server-side implementa i tre controlli discussi in questa sezione in un'unica funzione:

Listing 6.2: Validazione del pacchetto lato server.

```
def server_validate(packet, max_bet=80, time_window=5.0):
```

```

    if packet["nonce"] in nonce_registry:
        return False, "REPLAY_ATTACK: nonce già utilizzato"
    if abs(time.time() - packet["timestamp"]) > time_window:
        return False, "TIMESTAMP_SCADUTO: pacchetto fuori
finestra"
    if packet["amount"] > max_bet:
        return False, f"BET_LIMIT_EXCEEDED: {packet['amount']}
> {max_bet}"
    nonce_registry.add(packet["nonce"])
    return True, "OK"

```

I tre controlli vengono eseguiti in sequenza e nell'ordine mostrato: il controllo sul nonce ha priorità perché uno scarto immediato del pacchetto già visto evita di eseguire inutilmente i controlli successivi su un pacchetto comunque da rigettare.

Architettura di sicurezza delle transazioni

Per mitigare replay attack e injection, ogni richiesta include un nonce univoco verificato lato server e un timestamp che deve rientrare in una finestra di validità configurabile, nell'implementazione descritta inferiore a cinque secondi. Questa doppia protezione copre due superfici di attacco distinte: il nonce impedisce la riproduzione di pacchetti validi indipendentemente dalla loro età, mentre il controllo sul timestamp impedisce l'invio differito di pacchetti legittimi accumulati durante una sessione precedente e rimandati in un momento più favorevole.

La generazione del nonce tramite `secrets.token_hex(8)` produce stringhe di 16 caratteri esadecimali, corrispondenti a 8 byte di entropia crittograficamente sicura. La probabilità di collisione su n nonce generati è approssimabile con il paradosso del compleanno a $n^2/2^{65}$, valore trascurabile per qualsiasi volume di traffico realistico. Il nonce registry, implementato come un insieme in memoria nel prototipo simulato, in un sistema in produzione richiederebbe una struttura persistente con politica di scadenza coordinata con la finestra temporale di validità dei pacchetti: nonce più vecchi del limite temporale possono essere rimossi senza compromettere la protezione anti-replay.

Logging immutabile e riconciliazione

Il logging immutabile chiude il ciclo di sicurezza. Ogni azione rilevante, autenticazione, puntata, distribuzione, esito della mano, eventuale alert del profiler, genera una voce che, una volta scritta, non può essere modificata né cancellata senza lasciare tracce verificabili. La non-modificabilità può essere garantita con diversi meccanismi tecnici: una catena di hash in cui ogni voce include il digest della precedente, un log su storage append-only, o la replica su sistemi controllati da soggetti distinti. L'obiettivo è

garantire che, in caso di contestazione o indagine, sia possibile ricostruire l'intera sequenza degli eventi con certezza forense.

La riconciliazione periodica sfrutta questa traccia: il saldo corrente di ogni utente può essere ricalcolato indipendentemente sommando tutti i movimenti registrati nel log, e qualsiasi discrepanza tra il saldo calcolato e quello memorizzato nel database di produzione segnala una possibile alterazione non autorizzata dei record. Questo meccanismo intercetta non solo tentativi di frode esterna, ma anche alterazioni interne al sistema, errori software o interventi non autorizzati da parte di personale con accesso privilegiato.

In sintesi, ogni transazione deve superare tre controlli prima di essere processata e registrata: *identità*, verificare chi sta inviando la richiesta tramite token di sessione valido, *integrità*, verificare che il contenuto della richiesta non sia stato alterato e rientri nei limiti configurati, *freschezza*, verificare che il pacchetto sia nuovo tramite nonce e che non sia stato trasmesso con eccessivo ritardo tramite timestamp. Questi tre controlli, presi insieme, coprono le principali classi di attacco sul protocollo di comunicazione.

6.3 Integrazione tra contromisure operative e sicurezza informatica

I due domini analizzati nelle sezioni precedenti, le contromisure operative sul gioco e la sicurezza dell'infrastruttura informatica, non sono indipendenti ma si intersecano in modo significativo. Un attaccante che ha compromesso il PRNG del server tramite seed prediction non ha bisogno di contare le carte: conosce già l'ordine esatto del mazzo e può giocare con vantaggio assoluto indipendentemente dalla penetrazione, dalle spread restrictions e dal `BehaviouralProfiler`. In questo scenario, le contromisure operative sono completamente inutili perché la vulnerabilità è a un livello più profondo dell'architettura.

Viceversa, un contatore che applica il sistema Hi-Lo su un sistema con CSPRNG correttamente implementato non beneficia di alcun vantaggio derivante dalla prevedibilità dell' RNG: l'unica fonte di informazione disponibile è la sequenza di carte già visibili al tavolo, che è esattamente il caso per cui le contromisure operative sono progettate. La separazione tra i due domini di attacco implica che un sistema di sicurezza completo debba affrontarli entrambi con misure specifiche, senza presumere che la solidità di un layer renda superfluo l'altro.

Questa integrazione si riflette anche nel logging immutabile: un alert del `BehaviouralProfiler`, anche qualora risultasse in seguito un falso positivo, deve essere registrato con lo stesso livello di dettaglio di un evento di sicurezza informatica, perché la traccia comportamentale del giocatore nel tempo può essere rilevante sia per

contestazioni contrattuali sia per analisi forensi successive. Il sistema di audit diventa in questo modo il punto di convergenza tra i due livelli di sicurezza, garantendo che nessun evento rilevante vada perduto indipendentemente dal layer in cui si manifesta.

Capitolo 7

Aspetti Etici, Normativi e Privacy

7.1 Legalità del conteggio delle carte

Il conteggio delle carte occupa una posizione giuridica peculiare: lecito sul piano penale, sanzionabile sul piano contrattuale. Questa distinzione è essenziale per valutare le implicazioni del sistema di rilevamento sviluppato nei capitoli precedenti.

Sul piano penale, il conteggio non configura né la fattispecie di truffa ai sensi dell'art. 640 c.p. né quella di frode informatica ai sensi dell'art. 640-ter c.p. Il giocatore non introduce artifici o raggiri, non altera il software, non accede abusivamente a sistemi altrui. Si limita a elaborare mentalmente le informazioni pubbliche disponibili al tavolo. È un'attività puramente intellettuale, non perseguibile penalmente in Italia né nelle principali giurisdizioni europee.

Sul piano contrattuale la situazione è diversa. All'apertura del conto il giocatore sottoscrive Termini e Condizioni che tipicamente riservano al gestore il diritto di sospendere o chiudere gli account con pattern di gioco non ricreativi. Il conteggio viene classificato come *Advantage Play*, una condotta che la piattaforma può sanzionare con la risoluzione unilaterale del contratto, senza che ciò costituisca un illecito. Il casinò non denuncia il giocatore: esercita la propria autonomia negoziale.

Questa distinzione ha un effetto diretto sul *BehaviouralProfiler*. Il sistema non produce una notizia di reato, non è concepito per questo, ma genera un alert interno di rischio commerciale. L'output è uno strumento di gestione del rapporto contrattuale, non un atto di accusa, e questo perimetro operativo definisce anche i limiti della sua legittimità giuridica.

7.2 GDPR e behavioral tracking

Il funzionamento del `BehaviouralProfiler` solleva questioni concrete in materia di protezione dei dati personali. La raccolta sistematica di timing decisionale, pattern di puntata e telemetria di sessione è un trattamento di dati personali ad alto impatto, soggetto al Regolamento UE 2016/679.

La prima questione riguarda la **base giuridica del trattamento**. Per i dati anagrafici e le transazioni la base è l'esecuzione del contratto, ma il tracking comportamentale finalizzato alla detection si appoggia tipicamente al legittimo interesse del titolare ai sensi dell'art. 6, par. 1, lett. f. Il gestore ha un interesse economico legittimo a prevenire condotte che erodono il proprio margine. Tuttavia questo interesse deve superare il test di bilanciamento del considerando 47 del GDPR: se il tracking diventa particolarmente pervasivo o viene esteso a finalità diverse dalla sicurezza, la base del legittimo interesse può risultare insufficiente.

La seconda questione è la **minimizzazione dei dati**, sancita dall'art. 5, par. 1, lett. c. Il profiler è progettato per raccogliere esclusivamente i segnali strettamente necessari alla finalità di detection, in coerenza con questo principio. Il confine critico è il riutilizzo degli stessi dati per finalità diverse, ad esempio per la profilazione commerciale o per indurre il giocatore a prolungare la sessione. Quel riutilizzo, in assenza di specifica informativa, costituirebbe una violazione del principio di limitazione della finalità.

La terza questione riguarda i **processi decisionali automatizzati**. Poiché l'alert prodotto da `is_suspicious()` può tradursi nella sospensione dell'account, l'art. 22 del GDPR è direttamente applicabile: l'interessato ha il diritto di non essere sottoposto a decisioni basate unicamente su trattamenti automatizzati che producano effetti giuridici o analoghi. Il sistema deve pertanto prevedere la possibilità di intervento umano, consentire al giocatore di esprimere la propria posizione e garantire la contestabilità della decisione. La coerenza tra questo requisito normativo e le scelte progettuali adottate viene discussa nella Sezione 7.5.

7.3 Fairness bilanciata

Il monitoraggio comportamentale automatico introduce una tensione che non si risolve né sul piano tecnico né su quello giuridico. Il gestore ha il diritto di proteggere i propri margini economici. Il giocatore ha il diritto di giocare nel rispetto delle regole, anche con una competenza superiore alla media, senza essere penalizzato per questo.

Il punto critico è quando il monitoraggio smette di essere uno strumento di prevenzione dell'illecito e diventa un filtro di accesso basato sull'abilità. Un sistema che penalizza un utente non perché viola le regole, ma perché le segue con eccessiva disciplina statistica, pone un problema di *fairness algoritmica* difficilmente ignorabile.

Il contatore non bara: ottimizza. La linea tra ottimizzazione e frode è definita contrattualmente dal gestore, ma la sua legittimità etica non è scontata, e sarebbe disonesto presentarla come tale.

Il problema dei falsi positivi, analizzato nella Sezione 5.3, acquisisce qui una dimensione ulteriore. Un giocatore metodico che non conta affatto, ma gioca con ritmo regolare e puntate disciplinate, può generare un profilo sovrapponibile a quello di un contatore. La sospensione ingiustificata del suo account non è solo un errore statistico: è una lesione concreta del suo diritto a fruire del servizio, con potenziali conseguenze sul piano della buona fede contrattuale.

Nella letteratura informatica questo problema è studiato sotto il nome di *algorithmic fairness* [9]: un sistema decisionale automatico è considerato equo se non produce esiti sistematicamente svantaggiosi per un gruppo di utenti in base a caratteristiche non rilevanti per la decisione. Nel contesto del **BehaviouralProfiler**, la caratteristica “non rilevante” è il semplice stile di gioco metodico: un utente penalizzato non perché conta le carte, ma perché gioca con regolarità, subisce un trattamento che viola questa nozione di equità. Calibrare le soglie non è quindi solo una scelta tecnica, ma una scelta di valore che il progettista del sistema deve assumere consapevolmente.

Il parallelo con i sistemi IDS della sicurezza informatica è qui utile. Anche nella gestione dei sistemi informativi aziendali il monitoraggio del comportamento degli utenti interni, *User Behavior Analytics*, richiede un bilanciamento tra protezione del sistema e rispetto della sfera privata. La prassi consolidata prevede che l’alert automatico non sia mai l’ultima istanza: esattamente come in un SOC l’output dell’IDS richiede la validazione di un analista prima di qualsiasi azione, nel gioco online l’alert del **BehaviouralProfiler** dovrebbe innescare una revisione soggettiva prima di procedere a sanzioni permanenti. Nessun algoritmo è in grado di sostituire il giudizio contestuale umano nei casi limite: questa non è una debolezza del sistema, è un dato di fatto.

La soluzione non è rinunciare al monitoraggio, che rimane uno strumento legittimo e necessario, ma progettarlo con **trasparenza**, **proporzionalità** e **reversibilità**. Trasparenza significa informare l’utente dell’esistenza del sistema nelle condizioni di servizio. Proporzionalità significa calibrare le soglie in modo da minimizzare i falsi positivi accettando un margine di rischio sui falsi negativi. Reversibilità significa garantire la possibilità di contestare e correggere una decisione automatica errata.

7.4 Gioco Responsabile e Obblighi degli Operatori

Il quadro normativo italiano sul gioco a distanza ha subito una profonda revisione con il Decreto Legislativo 25 marzo 2024, n. 41, che raccoglie in un unico testo la disciplina del gioco online e introduce obblighi più stringenti per i concessionari [12]. Tra i principi ordinamentali esplicitamente enunciati figurano la *promozione del gioco*

responsabile, intesa come prevenzione di forme di gioco anomale o suscettibili di generare dipendenza patologica, e la *trasparenza dell'offerta*, quale garanzia della piena conoscibilità delle regole da parte dell'utente.

Sul piano operativo, il decreto impone ai concessionari ADM l'adozione di specifiche misure minime di gioco responsabile. Tra gli strumenti obbligatori rientrano i sistemi di autoesclusione temporanea o permanente, il registro di auto-esclusione accessibile tramite autenticazione SPID e i messaggi informativi obbligatori sui rischi del gioco. Le Linee guida ADM del 2025 [6] introducono inoltre un alert automatico dopo 60 minuti di gioco continuativo e il blocco dei depositi oltre la soglia dei 100 euro senza conferma esplicita dell'utente.

Questo contesto normativo interseca direttamente il sistema di rilevamento sviluppato in questa tesi. Il *BehaviouralProfiler*, progettato per identificare i contatori di carte, raccoglie dati comportamentali che potrebbero in linea di principio essere riutilizzati per profilare anche i giocatori problematici: frequenza delle sessioni, progressione delle puntate, durata del gioco. Questa sovrapposizione non è accidentale: le stesse anomalie statistiche che rivelano un contatore, varianza delle puntate, ritmo di gioco inusualmente costante, possono segnalare anche un giocatore in stato di dipendenza o di alterazione emotiva.

La distinzione tra le due finalità, protezione economica dell'operatore e tutela della salute dell'utente, è però fondamentale sul piano etico e giuridico. Il riutilizzo degli stessi dati per entrambe le finalità senza una specifica base giuridica distinta costituirebbe una violazione del principio di limitazione della finalità sancito dall'art. 5 del GDPR, già discusso nella Sezione 7.2. Un sistema progettato responsabilmente dovrebbe mantenere una separazione architetturale netta tra i due moduli, ciascuno con la propria base giuridica, i propri responsabili e le proprie politiche di conservazione dei dati.

7.5 Coerenza tra Scelte Progettuali e Requisiti Normativi

I capitoli precedenti hanno descritto un sistema articolato su più livelli: rilevamento comportamentale tramite il *BehaviouralProfiler* (Capitolo 5), contromisure operative e sicurezza informatica (Capitolo 6), e vincoli etici e normativi (questo capitolo). Vale la pena chiudere il cerchio osservando come le scelte progettuali adottate non siano state dettate solo da ragioni tecniche, ma abbiano risposto in modo coerente ai requisiti normativi identificati.

La decisione di produrre un log di allerta per la valutazione umana, anziché un'espulsione automatica, non è solo una cautela tecnica contro i falsi positivi: è la risposta diretta all'art. 22 del GDPR, che vieta decisioni basate unicamente su

trattamenti automatizzati con effetti giuridici sull'interessato. La scelta di raccogliere soltanto tre indicatori, varianza dei tempi, varianza delle puntate e correlazione di Pearson, risponde al principio di minimizzazione dei dati dell'art. 5. Il logging immutabile descritto nella Sezione 6.2.3 garantisce la non-ripudiabilità richiesta dalle linee guida ADM per la tracciabilità delle sessioni.

Questa convergenza tra vincoli tecnici e obblighi normativi non è un caso isolato: è il risultato di un approccio di *privacy by design*, in cui i requisiti di protezione dei dati vengono incorporati nella progettazione del sistema fin dalle fasi iniziali, piuttosto che aggiunti a posteriori come adempimento formale. Questo approccio, esplicitamente richiesto dall'art. 25 del GDPR, rappresenta lo standard atteso per qualsiasi sistema di monitoraggio comportamentale operante nel settore del gioco a distanza.

Appendice A

Biometria Comportamentale: dal Modello all'Implementazione

BIOMETRIA COMPORTAMENTALE

Dalla Teoria all'Applicazione

Teoria, modellazione e implementazione

Un caso pratico: rilevare i contatori di carte nel blackjack online

Un contatore bannato può tornare con un nuovo account, ma non con un nuovo comportamento. Tempi di reazione, cadenza dei clic, ritmo di navigazione e pattern di interazione con il dispositivo restano riconoscibili. La biometria comportamentale si fonda su questo principio: l'identità non è solo ciò che un utente sa o possiede, ma anche come si comporta.

A.1 Fondamenti teorici

La biometria comportamentale analizza i pattern specifici con cui un individuo interagisce con un dispositivo digitale. A differenza della biometria fisica, che identifica *chi si è*, quella comportamentale identifica *come si fa qualcosa*, costruendo un profilo dinamico che si aggiorna continuamente durante la sessione.

I segnali su cui si basa sono eterogenei: il timing tra azioni, la traiettoria e la cadenza dei clic, il pattern di navigazione nell'interfaccia e il fingerprinting del dispositivo. Quest'ultimo consente di associare sessioni diverse allo stesso hardware anche in assenza di credenziali condivise, contrastando i tentativi di multi-accounting con profili completamente nuovi.

Nei sistemi ludici online questa tecnologia trova applicazioni dirette: rilevamento di account compromessi, distinzione tra giocatori reali e software automatizzati, individuazione di pratiche scorrette come lo *smurfing*¹, e verifica continua dell'identità senza interrompere la sessione. La differenza strutturale rispetto all'autenticazione classica basata su password o OTP² è che la verifica non avviene una sola volta al login ma si estende all'intera sessione in modo trasparente per l'utente.

Il fenomeno del *behavioural drift*, ovvero la variazione nel tempo del profilo comportamentale per effetto di apprendimento, stress o cambio di dispositivo, impone ai sistemi reali di aggiornare continuamente il modello di baseline senza perdere la sensibilità necessaria a distinguere un'evoluzione naturale da un'anomalia fraudolenta, come già discusso nella Sezione 5.2.

A.2 Il concetto di ritmo comportamentale

Prima di definire il vettore di feature è utile formalizzare cosa si intende per *ritmo* nel contesto della biometria comportamentale, perché è un concetto più ricco di quanto la sola varianza possa catturare.

Il ritmo non è la velocità media con cui un utente agisce né la semplice dispersione dei tempi attorno a quella media. È la struttura temporale delle azioni: come si susseguono, se formano pattern ricorrenti, se rispondono in modo coerente agli eventi di gioco. Come tale, è un costrutto multidimensionale con almeno tre componenti distinte.

La *regolarità locale* misura in che misura il tempo di decisione alla mano n è predittivo di quello alla mano $n + 1$, formalmente l'autocorrelazione della serie temporale.

La *risposta contestuale* cattura il fatto che il ritmo umano non è stazionario: tende ad accelerare dopo una vincita e a rallentare dopo una perdita rilevante, e questa covariazione con gli eventi di gioco è parte integrante del profilo individuale.

La *stazionarietà nel tempo* indica se il ritmo si mantiene stabile nel corso della sessione o presenta derivate sistematiche.

La varianza T_{var} , adottata nel modello come proxy del ritmo, cattura solo la dispersione aggregata sull'intera sessione, risultando cieca rispetto all'ordine temporale dei valori e alla loro relazione con il contesto. Due sequenze con varianza identica possono avere andamenti radicalmente diversi: si tratta quindi di una semplificazione sufficiente a fini dimostrativi, le cui implicazioni sui falsi positivi sono discusse nella Sezione 5.3.

¹Pratica fraudolenta in cui giocatori esperti creano nuovi account per competere contro principianti.

²One-Time Password: password temporanea valida per una singola sessione, usata come secondo fattore di autenticazione.

A.3 Il vettore di feature

Il vettore adottato è $V = [T_{\text{avg}}, T_{\text{var}}, B_{\text{pattern}}, S_{\text{cons}}]$.

Tempo medio di decisione (T_{avg}). Misura il tempo tra la distribuzione dell'ultima carta e l'azione del giocatore. Tempi molto bassi e costanti suggeriscono automazione; valori variabili sono compatibili con un comportamento soggetto a esitazioni e reazioni emotive. Si osserva inoltre una differenza sistematica tra azioni: il tempo per Hit tende a essere più basso, mentre Stand richiede spesso una valutazione esplicita della carta scoperta del banco.

Varianza del tempo di risposta (T_{var}). Una varianza molto bassa indica un comportamento proceduralizzato; una varianza più alta è tipica di chi alterna rapidità ed esitazione in funzione del contesto, soprattutto dopo vincite o perdite rilevanti.

Pattern di variazione della puntata (B_{pattern}). Descrive come cambia la puntata in relazione agli eventi. Un rapporto $\text{Bet}_n/\text{Bet}_{n-1} > 1$ dopo una perdita può segnalare *chasing*³ o logiche di tipo Martingala⁴; lo stesso rapporto dopo una vincita può indicare una strategia di *press*⁵. Nel conteggio delle carte questo indicatore è particolarmente informativo se letto insieme a T_{var} : una puntata che cresce in modo coerente e ripetibile, accompagnata da tempi stabili, costituisce una firma più significativa di un semplice incremento dell'importo scommesso.

Consistenza comportamentale tra sessioni (S_{cons}). Misura se il profilo rimane stabile nel tempo, osservando la variazione del tempo medio di decisione tra sessioni consecutive (ΔT_{avg}) e della puntata media (ΔB_{avg}). Un comportamento molto stabile può indicare un profilo fortemente proceduralizzato; variazioni brusche sono compatibili con un giocatore occasionale o in stato di agitazione emotiva.

A titolo illustrativo, il vettore $V = [4.2 \text{ s}, 1.5 \text{ s}, 1.8\times, 0.9]$ descrive un profilo con tempo medio plausibilmente umano, varianza non meccanica, incremento marcato della puntata e buona stabilità tra sessioni. Preso isolatamente non è sufficiente per una classificazione, ma integrato con gli altri segnali della sessione diventa un indicatore utile.

Va precisato che il prototipo implementato nel **BehaviouralProfiler** (Capitolo 5) realizza un sottoinsieme di questo vettore: T_{var} è calcolato direttamente, B_{pattern} è approssimato dalla varianza delle puntate e dalla correlazione di Pearson con il True Count, mentre T_{avg} e S_{cons} non sono implementati nella versione attuale. Questa semplificazione è sufficiente ai fini dimostrativi, come discusso nella sezione seguente.

³Comportamento impulsivo che consiste nell'aumentare le puntate per recuperare le perdite.

⁴Strategia che prevede il raddoppio della puntata dopo ogni perdita.

⁵Tecnica che consiste nell'aumentare la puntata dopo una vincita per sfruttare una scia positiva.

A.4 Implementazione dimostrativa

Il dimostratore implementa un sottoinsieme del vettore V definito nella sezione precedente, centrato sui due segnali più informativi: la varianza dei tempi di decisione (T_{var}) e la correlazione tra puntate e True Count (approssimazione di B_{pattern}). Si tratta di una scelta progettuale consapevole, non di una limitazione accidentale: i due indicatori sono sufficienti a separare statisticamente i profili nelle simulazioni condotte, come dimostrato dai risultati del Capitolo 5. L'implementazione completa della classe `BehaviouralProfiler`, il listato del metodo `get_anomaly_score()` e i risultati quantitativi su 500.000 mani simulate per ciascun profilo sono riportati nel Capitolo 5. Si ricorda che il contatore non viene isolato perché vince di più, la win rate è quasi identica, ma perché la sua firma comportamentale è statisticamente incompatibile con quella di un giocatore privo di accesso allo stato del mazzo.

A.5 Limiti e prospettive

I limiti operativi del sistema, inclusi i falsi positivi su giocatori metodici, la sensibilità ai dispositivi condivisi e le implicazioni sul piano del GDPR, sono analizzati rispettivamente nella Sezione 5.3 e nella Sezione 7.2.

Sul piano tecnico rimane aperta la limitazione strutturale di T_{var} come misura scalare cieca rispetto all'ordine temporale: una rappresentazione completa richiederebbe l'autocorrelazione della serie e la modellazione della risposta agli eventi di gioco.

Lo sviluppo naturale del prototipo è l'integrazione di classificatori supervisionati come SVM o Random Forest, addestrati su sessioni storiche etichettate, o di approcci non supervisionati basati su clustering. Questo consentirebbe di sostituire le soglie fisse con un confine decisionale adattivo, capace di seguire l'evoluzione del comportamento nel tempo senza ricalibrazione manuale.

Il principio applicato al blackjack non è circoscritto al settore ludico. Nel *banking* lo stesso approccio viene usato per rilevare *account takeover*⁶ durante sessioni di home banking. Nell'*healthcare* protegge l'accesso ai terminali clinici verificando che i ritmi operativi del personale non presentino anomalie. Nella *enterprise security* il monitoraggio comportamentale si inserisce in un'architettura *zero trust*⁷, dove la verifica dell'identità è continua e non limitata al solo momento del login.

⁶Frode in cui un malintenzionato prende il controllo dell'account di un utente legittimo.

⁷Modello di sicurezza basato sul principio “non fidarsi mai, verificare sempre”, con autenticazione continua di ogni utente e dispositivo.

Bibliografia

- [1] R.R. Baldwin, W.E. Cantey, H. Maisel, J.P. McDermott, *The Optimum Strategy in Blackjack*, Journal of the American Statistical Association, vol. 51, n. 275, pp. 429–439, 1956.
- [2] E.O. Thorp, *Beat the Dealer*, 2a ed., New York, Vintage Books, 1966.
- [3] H. Dubner, *A Winning Strategy for Blackjack*, presentazione al Fall Joint Computer Conference, Las Vegas, 1963. Citato in: D. Schlesinger, *Blackjack Attack: Playing the Pros' Way*, 3a ed., Oakland, RGE Publishing, 2005.
- [4] S. Wong, *Professional Blackjack*, Pi Yee Press, 1994.
- [5] P.A. Griffin, *The Theory of Blackjack*, 6a ed., Las Vegas, Huntington Press, 1999.
- [6] D. Schlesinger, *Blackjack Attack: Playing the Pros' Way*, 3a ed., Oakland, RGE Publishing, 2005.
- [7] J.L. Kelly Jr., *A New Interpretation of Information Rate*, Bell System Technical Journal, vol. 35, n. 4, pp. 917–926, 1956.
- [8] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2a ed., Hillsdale, Lawrence Erlbaum Associates, 1988.
- [9] S. Barocas, M. Hardt, A. Narayanan, *Fairness and Machine Learning: Limitations and Opportunities*, fairmlbook.org, 2019.
- [10] K. Scarfone, P. Mell, *Guide to Intrusion Detection and Prevention Systems (IDPS)*, NIST Special Publication 800-94, National Institute of Standards and Technology, 2007.
- [11] Decreto Legislativo 25 marzo 2024, n. 41, *Riordino del settore dei giochi a partire da quelli a distanza*, Gazzetta Ufficiale della Repubblica Italiana, Serie Generale n. 76, 1 aprile 2024.

- [12] Decreto Legislativo 25 marzo 2024, n. 41, *Riordino del settore dei giochi a partire da quelli a distanza*, Gazzetta Ufficiale della Repubblica Italiana, Serie Generale n. 76, 1 aprile 2024.
- [13] Regolamento (UE) 2016/679 del Parlamento Europeo e del Consiglio, del 27 aprile 2016, relativo alla protezione delle persone fisiche con riguardo al trattamento dei dati personali, nonché alla libera circolazione di tali dati e che abroga la direttiva 95/46/CE (Regolamento generale sulla protezione dei dati), Gazzetta Ufficiale dell'Unione Europea, L 119, 4 maggio 2016.

Sitografia

- [1] S. Barocas, M. Hardt, A. Narayanan, *Fairness and Machine Learning* [risorsa online], <https://fairmlbook.org>, consultato luglio 2025.
- [2] K. Scarfone, P. Mell, *Guide to Intrusion Detection and Prevention Systems (IDPS)*, NIST SP 800-94 [risorsa online], <https://doi.org/10.6028/NIST.SP.800-94>, consultato luglio 2025.
- [3] *Decreto Legislativo 25 marzo 2024, n. 41* [risorsa online], <https://www.normattiva.it/uri-res/N2Ls?urn:nir:stato:decreto.legislativo:2024-03-25;41>, consultato luglio 2025.
- [4] *Decreto Legislativo 25 marzo 2024, n. 41* [risorsa online], <https://www.normattiva.it/uri-res/N2Ls?urn:nir:stato:decreto.legislativo:2024-03-25;41>, consultato luglio 2025.
- [5] *Regolamento (UE) 2016/679 (GDPR)* [risorsa online], <https://eur-lex.europa.eu/legal-content/IT/TXT/?uri=CELEX:32016R0679>, consultato luglio 2025.
- [6] Agenzia delle Dogane e dei Monopoli, *Linee guida per organismi di verifica inerenti alle regole tecniche relative al rapporto di concessione per l'esercizio e la raccolta dei giochi* [risorsa online], Determinazione Direttoriale n. 336762 del 16 giugno 2025, <https://www.adm.gov.it/portale/documents/20182/208320700/336762-16062025-PROROGA-TECNICA-CONCESSIONI-RACCOLTA-GIOCO-A-DISTANZA.pdf>, consultato luglio 2025.
- [7] M. Shackelford, *Wizard of Odds — Blackjack* [risorsa online], <https://wizardofodds.com/games/blackjack/>, consultato luglio 2025.
- [8] Blackjack Apprenticeship, *Hi-Lo System Guide for Card Counting in Blackjack* [risorsa online], <https://www.blackjackapprenticeship.com/hi-lo-system-guide-for-card-counting-blackjack/>, consultato luglio 2025.

Ringraziamenti

asdjhgtry.